

人工神经网络 (ANN) 超详细学习笔记

本笔记综合了神经网络基础知识、激活函数、参数初始化、损失函数、优化方法、正则化策略以及完整的分类案例，内容涵盖原理、数学公式、代码示例和调优建议，力求全面、深入、实用。

第一部分：神经网络概述与基本结构

1. 什么是人工神经网络

人工神经网络 (Artificial Neural Network, ANN) 是一种模仿生物神经网络结构和功能的计算模型。生物大脑由大量神经元相互连接而成，每个神经元接收来自树突的输入信号，在细胞体内进行加权积累，当电位超过阈值时，通过轴突输出电信号。人工神经网络借鉴了这一机制，将输入信号进行加权求和，再经过激活函数产生输出，从而模拟复杂的非线性映射关系。

人工神经网络是机器学习中的一个重要模型，尤其在深度学习领域中得到广泛应用。它由多个互相连接的人工神经元（也称为节点）构成，可以用于处理和学习复杂的数据模式，尤其适合解决非线性问题。

2. 神经网络的核心概念

2.1 神经元 (Neuron)

神经网络的基本单元是神经元。一个神经元接收多个输入 $x_1, x_2, \dots, x_n$ ，每个输入对应一个权重 $w_1, w_2, \dots, w_n$ ，还有一个偏置项 b 。神经元首先计算加权和：

$$z = \sum_{i=1}^n w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$

然后将 z 通过一个激活函数 f 得到输出：

$$a = f(z)$$

这个输出 a 会传递给下一层的神经元。这个过程就像来自不同树突（每个树突有不同的权重）的信息进行加权计算，输入到细胞中做加和，再通过激活函数输出细胞值。

同一层的多个神经元可以看作是通过并行计算来处理相同的输入数据，学习输入数据的不同特征。每个神经元可能会关注输入数据中的不同部分，从而捕捉到数据的不同属性。

2.2 神经网络的结构

一个典型的前馈全连接神经网络包含三层：

- **输入层 (Input Layer)**：即输入 x 的那一层（如图像、文本、声音等）。每个输入特征对应一个神经元。输入层将数据传递给下一层的神经元。
- **输出层 (Output Layer)**：即输出 y 的那一层。输出层的神经元根据网络的任务（回归、分类等）生成最终的预测结果。
- **隐藏层 (Hidden Layers)**：输入层和输出层之间都是隐藏层，神经网络的“深度”通常由隐藏层的数量决定。隐藏层的神经元通过加权和激活函数处理输入，并将结果传递到下一层。

全连接 (Fully Connected) 的含义：

- 同一层的神经元之间没有连接。
- 第 N 层的每个神经元与第 $N - 1$ 层的所有神经元相连。
- 全连接神经网络接收的样本数据是二维的，数据在每一层之间需要以二维的形式传递。
- 第 $N - 1$ 层神经元的输出就是第 N 层神经元的输入。
- 每个连接都有一个权重值（ w 系数和 b 系数）。

2.3 内部状态值与激活值

在神经网络的前向传播过程中，每个神经元会产生两个关键值：

- **内部状态值** $z = \mathbf{w}^T \mathbf{x} + b$ ：加权求和结果，反映了当前神经元接收到的输入、历史信息以及网络内部的权重计算结果。
- **激活值** $a = f(z)$ ：经过激活函数后的输出。

在反向传播过程中，还会计算梯度：

- 激活值的梯度 $\frac{\partial L}{\partial a}$
- 内部状态值的梯度 $\frac{\partial L}{\partial z}$

通过控制每个神经元的内部状态值、激活值的大小；每一层的内部状态值的方差、每一层的激活值的方差可让整个神经网络工作的更好。

2.4 神经网络的参数量计算

对于全连接层，如果输入维度为 d_{in} ，输出维度为 d_{out} ，则该层的参数量为：

$$\text{参数个数} = d_{in} \times d_{out} \quad (\text{权重}) + d_{out} \quad (\text{偏置})$$

例如：输入层 20 个神经元，第一隐藏层 128 个神经元，则第一隐藏层的参数量 = $20 \times 128 + 128 = 2688$ 。

3. 神经网络的前向传播与反向传播简述

- **前向传播**：数据从输入层流向输出层，逐层计算 z 和 a ，最终得到预测值。
- **反向传播**：根据损失函数计算输出层的误差，然后利用链式法则逐层向后传播梯度，计算每个权重和偏置对损失的贡献，用于参数更新。

PyTorch 的 `autograd` 模块自动完成反向传播，无需手动推导。

第二部分：激活函数详解

激活函数是神经网络中引入**非线性因素**的关键组件。如果没有激活函数，无论网络有多少层，最终都等价于一个线性模型，无法拟合复杂的非线性关系。激活函数通过对每层的输出数据进行变换，进而为整个网络注入非线性因素，使神经网络能够拟合各种曲线。我们的网络参数在更新时使用反向传播算法（BP），这就要求激活函数必须可微。

1. 网络非线性因素理解

没有引入非线性因素的网络等价于使用一个线性模型来拟合。通过给网络输出增加激活函数，实现引入非线性因素，使得网络模型可以逼近任意函数，提升网络对复杂问题的拟合能力。

• 2. 常见激活函数详解

- 2.1 Sigmoid 激活函数

公式：

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

导数：

$$\sigma'(x) = \sigma(x) \cdot (1 - \sigma(x))$$

函数图像特点：

- 输出范围：(0, 1)，可以理解为概率。
- 光滑且连续，处处可导。
- 以 (0, 0.5) 为中心对称。

从 Sigmoid 函数图像可以得到，Sigmoid 函数可以将任意的输入映射到 (0,1) 之间。当输入的值大致在 < -6 或者 > 6 时，意味着输入任何值得到的激活值都是差不多的，这样会丢失部分的信息。比如：输入 100 和输入 10000 经过 Sigmoid 的激活值几乎都是等于 1 的，但是输入的数据之间相差 100 倍的信息就丢失了。对于 Sigmoid 函数而言，输入值在 $[-6, 6]$ 之间输出值才会有明显差异，输入值在 $[-3, 3]$ 之间才会有比较好的效果。

通过导数图像，我们发现导数数值范围是 (0,0.25)，当输入的值 < -6 或者 > 6 时，Sigmoid 激活函数图像的导数接近为 0，此时网络参数将更新极其缓慢，或者无法更新。一般来说，Sigmoid 网络在 5 层之内就会产生梯度消失现象。而且，该激活函数的激活值并不是以 0 为中心的，激活值总是偏向正数，导致梯度更新时，只会对某些特征产生相同方向的影响，所以在实践中这种激活函数使用的很少。

优点：

- 平滑、易于求导。
- 输出在 (0,1) 之间，适合作为概率输出（如二分类的输出层）。

缺点：

- **梯度饱和**：当输入绝对值很大时，梯度接近于 0，导致参数更新极慢（梯度消失）。
- 输出不是零中心（zero-centered），导致梯度更新效率低。
- 指数运算计算量较大。

适用场景：二分类问题的输出层（配合 `BCELoss`）。

代码示例与绘图：

```

import torch
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['Microsoft YaHei']
plt.rcParams['axes.unicode_minus'] = False

# 创建画布
_, axes = plt.subplots(1, 2, figsize=(12, 4))

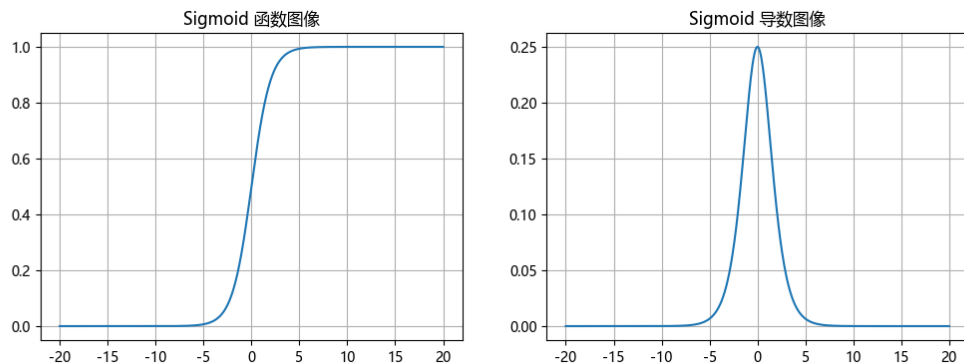
# 函数图像
x = torch.linspace(-20, 20, 1000)
y = torch.sigmoid(x)
axes[0].plot(x, y)
axes[0].grid()
axes[0].set_title('Sigmoid 函数图像')

# 导数图像
x = torch.linspace(-20, 20, 1000, requires_grad=True)
torch.sigmoid(x).sum().backward()
axes[1].plot(x.detach(), x.grad)
axes[1].grid()
axes[1].set_title('Sigmoid 导数图像')

plt.show()

```

绘图：



2.2 Tanh 激活函数

Tanh 叫做双曲正切函数。

公式：

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{1 - e^{-2x}}{1 + e^{-2x}}$$

导数：

$$\tanh'(x) = 1 - \tanh^2(x)$$

函数图像特点：

- 输出范围：(-1, 1)，**零中心**。
- 形状与 Sigmoid 相似，但输出包含负数。

由函数图像可以看到，Tanh 函数将输入映射到 (-1, 1) 之间，图像以 0 为中心，激活值在 0 点对称，当输入的值大概 ≤ -3 或者 > 3 时将被映射为 -1 或者 1。其导数值范围 (0, 1)，当输入的值大概 ≤ -3 或者 > 3 时，其导数近似 0。与 Sigmoid 相比，它是以 0 为中心的，使得其收敛速度要比 Sigmoid 快，减少迭代次数。然而，Tanh 两侧的导数也为 0，同样会造成梯度消失。

优点：

- 输出零中心，有利于梯度下降的收敛速度。
- 比 Sigmoid 的梯度饱和区域稍宽。

缺点：

- 仍存在梯度饱和问题（当输入绝对值较大时，梯度接近 0）。
- 计算量稍大（指数运算）。

适用场景：隐藏层（但通常不如 ReLU 常用），尤其是需要零中心输出的场合。若使用时可在隐藏层使用 Tanh 函数，在输出层使用 Sigmoid 函数。

代码示例与绘图：

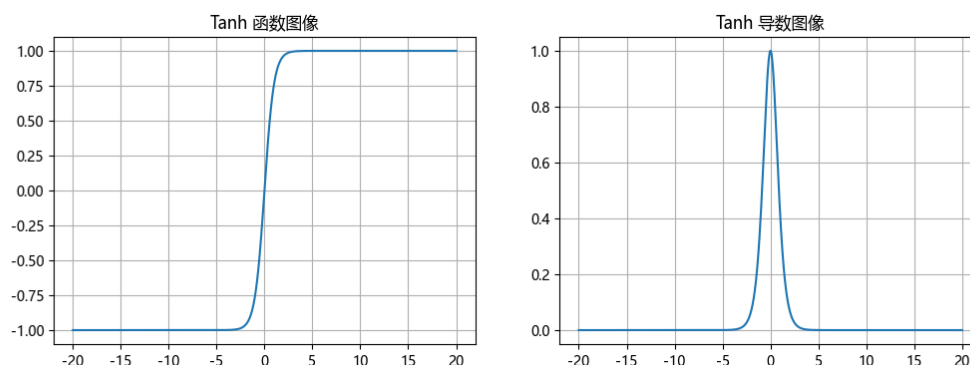
```
_, axes = plt.subplots(1, 2, figsize=(12, 4))

x = torch.linspace(-20, 20, 1000)
y = torch.tanh(x)
axes[0].plot(x, y)
axes[0].grid()
axes[0].set_title('Tanh 函数图像')

x = torch.linspace(-20, 20, 1000, requires_grad=True)
torch.tanh(x).sum().backward()
axes[1].plot(x.detach(), x.grad)
axes[1].grid()
axes[1].set_title('Tanh 导数图像')

plt.show()
```

绘图：



- 2.3 ReLU 激活函数 (Rectified Linear Unit)

公式：

$$\text{ReLU}(x) = \max(0, x)$$

导数：

$$\text{ReLU}'(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$$

函数图像特点：

- 负半轴输出为 0，正半轴线性输出。
- 在 $x = 0$ 处不可导（但在实现中通常取左导数或右导数）。

ReLU 激活函数将小于 0 的值映射为 0，而大于 0 的值则保持不变，它更加重视正信号，而忽略负信号，这种激活函数运算更为简单，能够提高模型的训练效率。当 $x < 0$ 时，ReLU 导数为 0，而当 $x > 0$ 时，则不存在饱和问题。所以，ReLU 能够在 $x > 0$ 时保持梯度不衰减，从而缓解梯度消失问题。然而，随着训练的推进，部分输入会落入小于 0 区域，导致对应权重无法更新。这种现象被称为“神经元死亡”。

ReLU 是目前最常用的激活函数。与 Sigmoid 相比，ReLU 的优势是：

- 采用 Sigmoid 函数，计算量大（指数运算），反向传播求误差梯度时，计算量相对大；而采用 ReLU 激活函数，整个过程的计算量节省很多。
- Sigmoid 函数反向传播时，很容易就会出现梯度消失的情况，从而无法完成深层网络的训练；而采用 ReLU 激活函数，当输入的值 > 0 时，梯度为 1，不会出现梯度消失的情况。
- ReLU 会使一部分神经元的输出为 0，这样就造成了网络的稀疏性，并且减少了参数的相互依存关系，缓解了过拟合问题的发生。

优点：

- **计算简单**：只需比较和取最大值，没有指数运算。
- **缓解梯度消失**：正半轴梯度恒为 1。

- **稀疏激活**：负半轴输出为 0，使部分神经元不激活，增加模型稀疏性。
- 实际训练中收敛速度快。

缺点：

- **Dead ReLU 问题**：如果某个神经网络的所有输入都是负数，那么它的梯度为 0，参数永远不会更新，导致神经元“死亡”。通常由学习率过高或权重初始化不当引起。
- 输出不是零中心。

改进版本（Leaky ReLU / PReLU / ELU）：

- **Leaky ReLU**：负半轴有一个小的正斜率，如 $\text{LeakyReLU}(x) = \max(0.01x, x)$ ，解决 Dead ReLU。
- **PReLU**：负半轴的斜率作为可学习参数。
- **ELU**：负半轴使用指数函数，输出接近零中心。

适用场景：隐藏层的默认首选，除非有特殊问题。

代码示例与绘图：

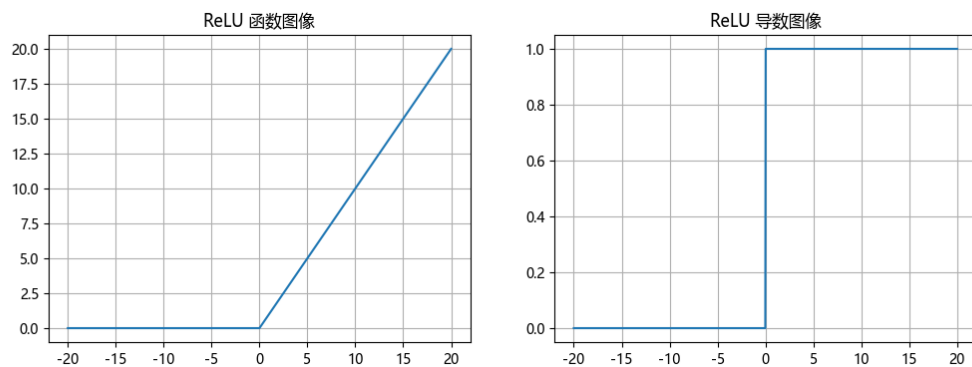
```
_, axes = plt.subplots(1, 2, figsize=(12, 4))

x = torch.linspace(-20, 20, 1000)
y = torch.relu(x)
axes[0].plot(x, y)
axes[0].grid()
axes[0].set_title('ReLU 函数图像')

x = torch.linspace(-20, 20, 1000, requires_grad=True)
torch.relu(x).sum().backward()
axes[1].plot(x.detach(), x.grad)
axes[1].grid()
axes[1].set_title('ReLU 导数图像')

plt.show()
```

绘图：



2.4 Softmax 激活函数

Softmax 用于多分类过程中，它是二分类函数 Sigmoid 在多分类上的推广，目的是将多分类的结果以概率的形式展现出来。

公式：

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}, \quad i = 1, 2, \dots, C$$

其中 C 是类别总数。

特点：

- 输出值在 (0, 1) 之间。
- 所有类别的输出概率之和为 1。
- 保持原有大小顺序（单调递增）。

Softmax 就是将网络输出的 logits 通过 Softmax 函数，映射成为 (0,1) 的值，而这些值的累和为 1（满足概率的性质），那么我们将它理解成概率，选取概率最大（也就是值对应最大的）节点，作为我们的预测目标类别。

优点：

- 输出具有概率解释。
- 与交叉熵损失结合时梯度形式简单。

缺点：

- 计算涉及指数，可能产生数值上溢（可用减去最大值稳定化）。
- 对异常值敏感。

适用场景：多分类问题的输出层。

代码示例：

```
scores = torch.tensor([0.2, 0.02, 0.15, 0.15, 1.3, 0.5, 0.06, 1.1,
0.05, 3.75])
probabilities = torch.softmax(scores, dim=0)
print(probabilities)
# tensor([0.0212, 0.0177, 0.0202, 0.0202, 0.0638, 0.0287, 0.0185,
0.0522, 0.0183, 0.7392])
```

• 3. 其他常见激活函数

下表列举了更多激活函数：

激活函数	公式	导数	值域
Identity	$f(x) = x$	$f'(x) = 1$	$(-\infty, \infty)$
Binary step	$f(x) = \{0 \text{ for } x < 0, 1 \text{ for } x \geq 0\}$	除 0 外为 0	$\{0, 1\}$
Logistic (Sigmoid)	$f(x) = \frac{1}{1+e^{-x}}$	$f(x)(1 - f(x))$	$(0, 1)$
TanH	$\tanh(x)$	$1 - f(x)^2$	$(-1, 1)$
ReLU	$\max(0, x)$	$\{0 \text{ for } x < 0, 1 \text{ for } x \geq 0\}$	$[0, \infty)$
Leaky ReLU	$\{0.01x \text{ for } x < 0, x \text{ for } x \geq 0\}$	$\{0.01 \text{ for } x < 0, 1 \text{ for } x \geq 0\}$	$(-\infty, \infty)$
PReLU	$\{\alpha x \text{ for } x < 0, x \text{ for } x \geq 0\}$	$\{\alpha \text{ for } x < 0, 1 \text{ for } x \geq 0\}$	$(-\infty, \infty)$
RReLU	同上, α 随机	同上	$(-\infty, \infty)$
ELU	$\{\alpha(e^x - 1) \text{ for } x < 0, x \text{ for } x \geq 0\}$	$\{f(x) + \alpha \text{ for } x < 0, 1 \text{ for } x \geq 0\}$	$(-\infty, \infty)$

• 4. 激活函数选择指南

对于隐藏层：

1. 优先选择 ReLU 激活函数。
2. 如果 ReLU 效果不好，那么尝试其他激活，如 Leaky ReLU 等。
3. 如果你使用了 ReLU，需要注意一下 Dead ReLU 问题，避免出现 0 梯度从而导致过多的神经元死亡。
4. 少使用 Sigmoid 激活函数，可以尝试使用 Tanh 激活函数。

对于输出层：

1. 二分类问题选择 Sigmoid 激活函数。

2. 多分类问题选择 Softmax 激活函数。
3. 回归问题选择 Identity 激活函数。

第三部分：参数初始化方法

参数初始化是神经网络训练前的关键步骤。良好的初始化可以加速收敛、避免梯度消失/爆炸，而不当的初始化则可能导致模型难以训练。PyTorch 的 `torch.nn.init` 模块提供了多种初始化方法。

1. 为什么参数初始化很重要？

在训练开始前，我们需要为模型的权重 W 和偏置 b 赋予初始值。如果初始化不当，可能出现：

- **梯度消失**：权重过小，激活值逐渐趋近于 0，反向传播时梯度指数级衰减。
- **梯度爆炸**：权重过大，激活值和梯度指数级增长，导致数值溢出。
- **对称性问题**：如果同一层的所有神经元初始化为相同值，它们会学习到相同的特征，丧失表达多样性。

因此，好的初始化应满足：

- 各层的激活值方差保持稳定，避免过小或过大。
- 打破对称性（通常使用随机初始化）。

参数初始化的具体作用：

- **防止梯度消失或爆炸**：初始权重值过大或过小会导致梯度在反向传播中指数级增大或缩小。
 - **提高收敛速度**：合理的初始化使得网络的激活值分布适中，有助于梯度高效更新。
 - **保持对称性破除**：权重的初始化需要打破对称性，否则网络的学习能力会受到限制。
-

• 2. 常见的初始化方法

- 2.1 均匀分布初始化 (Uniform Initialization)

从指定区间 $[a, b]$ 的均匀分布中随机采样。默认区间为 $(0, 1)$ ，但通常需要根据输入维度调整范围。可以设置为在 $(-\frac{1}{\sqrt{d}}, \frac{1}{\sqrt{d}})$ 均匀分布中生成当前神经元的权重，其中 d 为神经元的输入数量。

优点：能有效打破对称性。

缺点：随机选择范围不当可能导致梯度问题。

适用场景：浅层网络或低复杂度模型（隐藏层 1-3 层，总层数不超过 5 层）。

代码示例：

```
import torch
import torch.nn as nn

linear = nn.Linear(5, 3)
nn.init.uniform_(linear.weight, a=-0.1, b=0.1) # 从 [-0.1, 0.1] 均匀分布采样
print(linear.weight)
```

- 2.2 正态分布初始化 (Normal Initialization)

从指定均值和标准差的正态分布中随机采样。常用的简单版本是均值为 0、标准差为 0.01 或 0.001 的分布。

优点：能有效打破对称性。

缺点：随机选择范围不当可能导致梯度问题。

适用场景：浅层网络或低复杂度模型。

代码示例：

```
nn.init.normal_(linear.weight, mean=0.0, std=0.01)
```

- 2.3 常数初始化 (Constant Initialization)

所有权重初始化为同一个常数。通常用于偏置的初始化（例如将偏置初始化为 0 或小数）。

- **全 0 初始化**: `nn.init.zeros_(linear.weight)`
- **全 1 初始化**: `nn.init.ones_(linear.weight)`
- **固定值初始化**: `nn.init.constant_(linear.weight, 0.5)`

全 0 初始化: 优点: 实现简单。缺点: 无法打破对称性, 所有神经元更新方向相同, 无法有效训练。适用场景: 几乎不使用, 仅用于偏置项的初始化。

全 1 初始化: 优点: 实现简单。缺点: 无法打破对称性, 所有神经元更新方向相同, 无法有效训练; 会导致激活值在网络中呈指数增长, 容易出现梯度爆炸。适用场景: 测试或调试 (验证神经网络是否能正常前向传播和反向传播); 特殊模型结构; 偏置初始化 (偶尔可以将偏置初始化为小的正值如 0.1, 但很少用 1 作为偏置的初始值)。

固定值初始化: 优点: 实现简单。缺点: 无法打破对称性; 初始权重过大或过小可能导致梯度爆炸或梯度消失。适用场景: 测试或调试。

注意: 不能将权重全部初始化为相同值, 否则同一层的神经元将无法学习到不同的特征。常数初始化通常只用于偏置或特定层 (如 BN 的 `weight` 初始化为 1, `bias` 初始化为 0)。

- 2.4 Xavier / Glorot 初始化

Xavier 初始化 (由 Glorot 和 Bengio 在 2010 年提出) **适用于 Sigmoid 和 Tanh 等饱和激活函数**。其目标是使前向传播和反向传播时, 各层激活值的方差保持一致。

原理: 假设输入均值为 0, 希望输出的方差与输入的方差相等。推导得出权重应从以下分布中采样:

- **正态分布**: $W \sim \mathcal{N}(0, \text{std}^2)$, 其中 $\text{std} = \sqrt{\frac{2}{\text{fan_in} + \text{fan_out}}}$
- **均匀分布**: $W \sim U(-\text{limit}, \text{limit})$, 其中 $\text{limit} = \sqrt{\frac{6}{\text{fan_in} + \text{fan_out}}}$

其中:

- `fan_in`: 该层输入神经元的个数
- `fan_out`: 该层输出神经元的个数

优点: 适用于 Sigmoid、Tanh 等激活函数, 解决梯度消失问题。

缺点: 对 ReLU 等激活函数表现欠佳。

适用场景: 深度网络 (10 层及以上), 使用 Sigmoid 或 Tanh 激活函数。

代码示例：

```
# 正态分布 Xavier
nn.init.xavier_normal_(linear.weight)

# 均匀分布 Xavier
nn.init.xavier_uniform_(linear.weight)
```

- 2.5 Kaiming / He 初始化

Kaiming 初始化（由何恺明等人在 2015 年提出）**专为 ReLU 及其变体（Leaky ReLU、PReLU）设计**。由于 ReLU 会将一半的神经元输出置为 0，导致方差减半，因此需要调整方差尺度。

公式（对于 ReLU）：

- 正态分布： $W \sim \mathcal{N}(0, \text{std}^2)$ ，其中 $\text{std} = \sqrt{\frac{2}{\text{fan_in}}}$
- 均匀分布： $W \sim U(-\text{limit}, \text{limit})$ ，其中 $\text{limit} = \sqrt{\frac{6}{\text{fan_in}}}$

优点：适合 ReLU，能保持梯度稳定。

缺点：对非 ReLU 激活函数效果一般。

适用场景：深度网络（10 层及以上），使用 ReLU、LeakyReLU 激活函数。

代码示例：

```
# Kaiming 正态分布（He 初始化）
nn.init.kaiming_normal_(linear.weight, mode='fan_in',
nonlinearity='relu')

# Kaiming 均匀分布
nn.init.kaiming_uniform_(linear.weight, mode='fan_in',
nonlinearity='relu')
```

- `mode='fan_in'`：保持前向传播的方差稳定（推荐）。
- `mode='fan_out'`：保持反向传播的方差稳定。
- `nonlinearity`：可选 `'relu'` 或 `'leaky_relu'`，用于调整增益。

• 3. 偏置的初始化

偏置通常初始化为 0 或小常数。在分类任务的输出层，偏置可初始化为某一类别的先验概率的对数（不常用）。

```
# 偏置初始化为 0
nn.init.zeros_(linear.bias)

# 偏置初始化为小常数（如 0.01）
nn.init.constant_(linear.bias, 0.01)
```

• 4. 初始化方法的选择策略

激活函数	推荐初始化方法
Sigmoid / Tanh	Xavier (Glorot) 初始化
ReLU / Leaky ReLU / PReLU	Kaiming (He) 初始化
线性层（无激活）	Xavier 或 Kaiming 均可
深层网络	使用 Xavier / Kaiming，避免简单高斯初始化（如 $N(0,0.01)$ ）
迁移学习（微调）	使用预训练模型的权重，不需要随机初始化

PyTorch 默认初始化：大多数层（如 `nn.Linear`）默认使用 Kaiming 均匀分布初始化，适合 ReLU。

• 5. 完整示例：自定义网络的参数初始化

以下示例定义了一个两层网络，分别对不同的隐藏层使用不同的初始化方法：

```
import torch
import torch.nn as nn

class MyModel(nn.Module):
    def __init__(self, input_dim=20, hidden_dim=128, output_dim=4):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim) # 第一隐藏层
        self.fc2 = nn.Linear(hidden_dim, hidden_dim) # 第二隐藏层
        self.out = nn.Linear(hidden_dim, output_dim) # 输出层
```

```

# 自定义初始化
self._initialize_weights()

def _initialize_weights(self):
    # 第一层: Xavier 均匀分布 (假设使用 Tanh)
    nn.init.xavier_uniform_(self.fc1.weight)
    nn.init.zeros_(self.fc1.bias)

    # 第二层: Kaiming 正态分布 (使用 ReLU)
    nn.init.kaiming_normal_(self.fc2.weight, mode='fan_in',
nonlinearity='relu')
    nn.init.zeros_(self.fc2.bias)

    # 输出层: 简单的正态分布 (小标准差)
    nn.init.normal_(self.out.weight, mean=0.0, std=0.01)
    nn.init.zeros_(self.out.bias)

def forward(self, x):
    x = torch.tanh(self.fc1(x))    # 第一层用 Tanh
    x = torch.relu(self.fc2(x))    # 第二层用 ReLU
    x = self.out(x)               # 输出层无激活 (后续加 Softmax)
    return x

model = MyModel()

```

• 6. 初始化方法总结表

初始化方法	分布	公式 (方差/范围)	适用激活函数
Xavier 正态	正态	$\text{std} = \sqrt{2/(\text{fan_in} + \text{fan_out})}$	Sigmoid, Tanh
Xavier 均匀	均匀	$\text{limit} = \sqrt{6/(\text{fan_in} + \text{fan_out})}$	Sigmoid, Tanh
Kaiming 正态	正态	$\text{std} = \sqrt{2/\text{fan_in}}$	ReLU, Leaky ReLU
Kaiming 均匀	均匀	$\text{limit} = \sqrt{6/\text{fan_in}}$	ReLU, Leaky ReLU
普通正态	正态	固定 std (如 0.01)	浅层网络
常数	恒定	固定值	偏置, 或特殊情况

第四部分：损失函数详解

损失函数 (Loss Function) 用于衡量模型预测值与真实值之间的差异。训练神经网络的过程就是通过优化算法不断减小损失函数值的过程。不同任务类型 (分类、回归) 需要使用不同的损失函数。

1. 损失函数概述

在深度学习中，损失函数是用来衡量模型参数的质量的函数，衡量的方式是比较网络输出和真实输出的差异。损失函数在不同的文献中名称是不一样的，主要有以下几种命名方式：

- **损失 (Loss)**：单个样本的误差。
- **代价 (Cost Function)**：整个训练集上的平均损失。
- **目标函数 (Objective Function)**：优化问题的最终函数，通常为代价 + 正则化项。
- **误差函数 (Error Function)**：同损失函数。

损失函数作用：

- 评估性能：反映模型预测结果与目标值的匹配程度。
- 指导优化：通过梯度下降等算法最小化损失函数，优化模型参数。

PyTorch 的 `torch.nn` 模块提供了丰富的损失函数实现，它们都继承自 `nn.Module`。

2. 分类任务损失函数

2.1 多分类交叉熵损失 (Cross-Entropy Loss)

在多分类任务通常使用 Softmax 将 logits 转换为概率的形式，所以多分类的交叉熵损失也叫做 Softmax 损失。它的计算方法是：

$$\text{Loss} = - \sum_i y_i \log(S(f(x)_i))$$

其中：

- y_i 是样本 x 属于某一个类别的真实概率 (通常 one-hot 编码)。
- $f(x)$ 是样本属于某一类别的预测分数 (logits)。
- S 是 Softmax 激活函数，将属于某一类别的预测分数转换成概率。

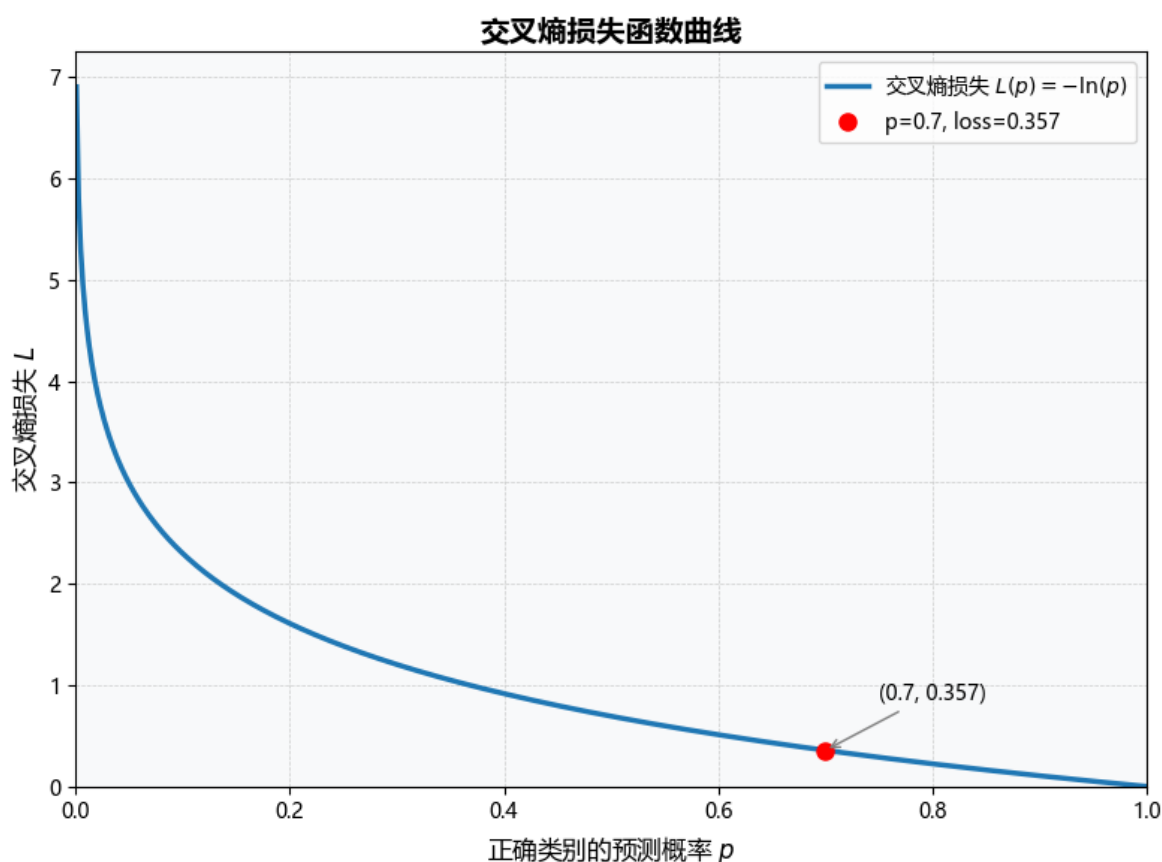
- L 用来衡量真实值 y 和预测值 $f(x)$ 之间差异性的损失结果。

对于单个样本，真实标签为 y （类别索引，非 one-hot），预测的 logits 为 $\hat{y}_1, \hat{y}_2, \dots, \hat{y}_C$ ，则损失为：

$$\text{Loss}(y, \hat{y}) = -\log \left(\frac{e^{\hat{y}_y}}{\sum_{j=1}^C e^{\hat{y}_j}} \right) = -\hat{y}_y + \log \left(\sum_{j=1}^C e^{\hat{y}_j} \right)$$

实际上，`nn.CrossEntropyLoss` 将 `LogSoftmax` 和 `Negative Log-Likelihood (NLLoss)` 合并为一个数值稳定的实现。

图例：



例子：上图中的交叉熵损失为：

$-(0 \log(0.10) + 1 \log(0.7) + 0 \log(0.2)) = -\log 0.7$ 。从概率角度理解，我们的目的是最小化正确类别所对应的预测概率的对数的负值（损失值最小）。

重要特性：

- 输入是 **未经 Softmax 的原始 logits**（形状 `(N, C)`），而不是概率。
- 真实标签是 **类别索引**（形状 `(N,)`），类型为 `torch.int 64`，取值范围 `[0, C-1]`。
- 输出为标量损失（如果 `reduction='mean'`）或每个样本的损失向量。

代码示例：

```
import torch
import torch.nn as nn

# 示例: 3 个样本, 4 个类别
logits = torch.tensor([[0.2, 0.6, 0.1, 0.1],
                       [0.8, 0.1, 0.05, 0.05],
                       [0.1, 0.2, 0.5, 0.2]], requires_grad=True)
targets = torch.tensor([1, 0, 2], dtype=torch.int64) # 真实类别索引

criterion = nn.CrossEntropyLoss() # reduction='mean' 默认
loss = criterion(logits, targets)
print(loss.item())
```

注意事项:

- 输入 logits 可以是任意实数，内部会进行数值稳定化处理（减去最大值）。
- 对于高维度输入（如图片分割），标签可以是形状 `(N, H, W)`，logits 形状 `(N, C, H, W)`，损失函数会自动处理。
- 多分类问题中，不需要手动对输出做 Softmax，直接使用 `CrossEntropyLoss` 即可。

- 2.2 二分类交叉熵损失 (Binary Cross-Entropy Loss)

对于二分类问题（输出只有一个概率值），使用 `nn.BCELoss`。该损失函数要求输入是经过 Sigmoid 的概率值（范围 `[0,1]`）。

数学公式:

$$\text{BCE}(y, \hat{y}) = -[y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})]$$

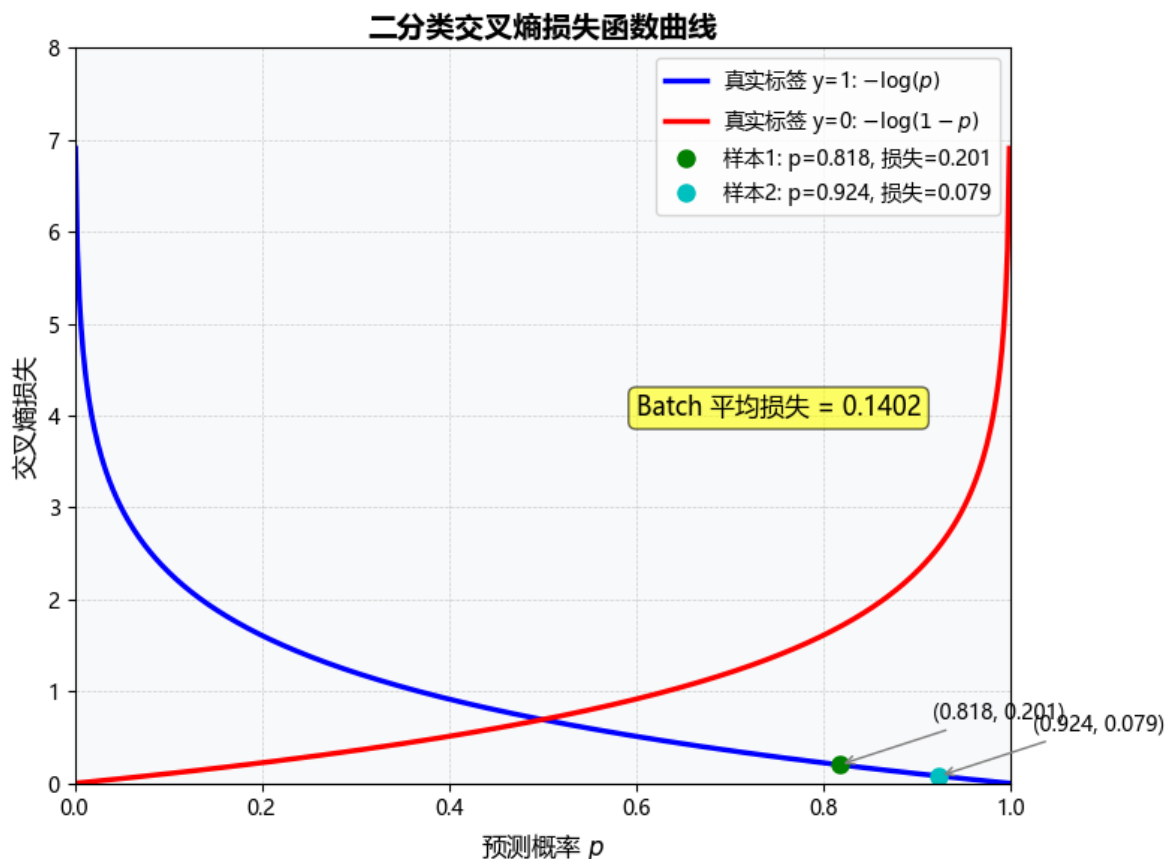
其中 y 为真实标签（0 或 1）， $\hat{y}$ 为预测概率。

代码示例:

```
# 预测概率（经过 Sigmoid），形状 (N,)
probabilities = torch.tensor([0.6901, 0.5459], requires_grad=True)
targets = torch.tensor([0., 1.], dtype=torch.float32)

criterion = nn.BCELoss()
loss = criterion(probabilities, targets)
print(loss.item())
```

图例:



数值稳定的版本： `nn.BCEWithLogitsLoss` 将 Sigmoid 和 BCE 合并，并在内部进行数值稳定处理，推荐使用。

```
criterion = nn.BCEWithLogitsLoss() # 输入是原始 logits
logits = torch.tensor([1.0, -0.5]) # 未经过 Sigmoid
targets = torch.tensor([1., 0.])
loss = criterion(logits, targets)
```

使用场景：

- 二分类问题的输出层（一个神经元）。
- 多标签分类（每个标签独立二分类）。

• 3. 回归任务损失函数

- 3.1 MAE 损失 (L1 Loss)

Mean Absolute Loss (MAE) 也被称为 L1 Loss，是以绝对误差作为距离。

公式：

$$\text{L1 Loss}(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

特点：

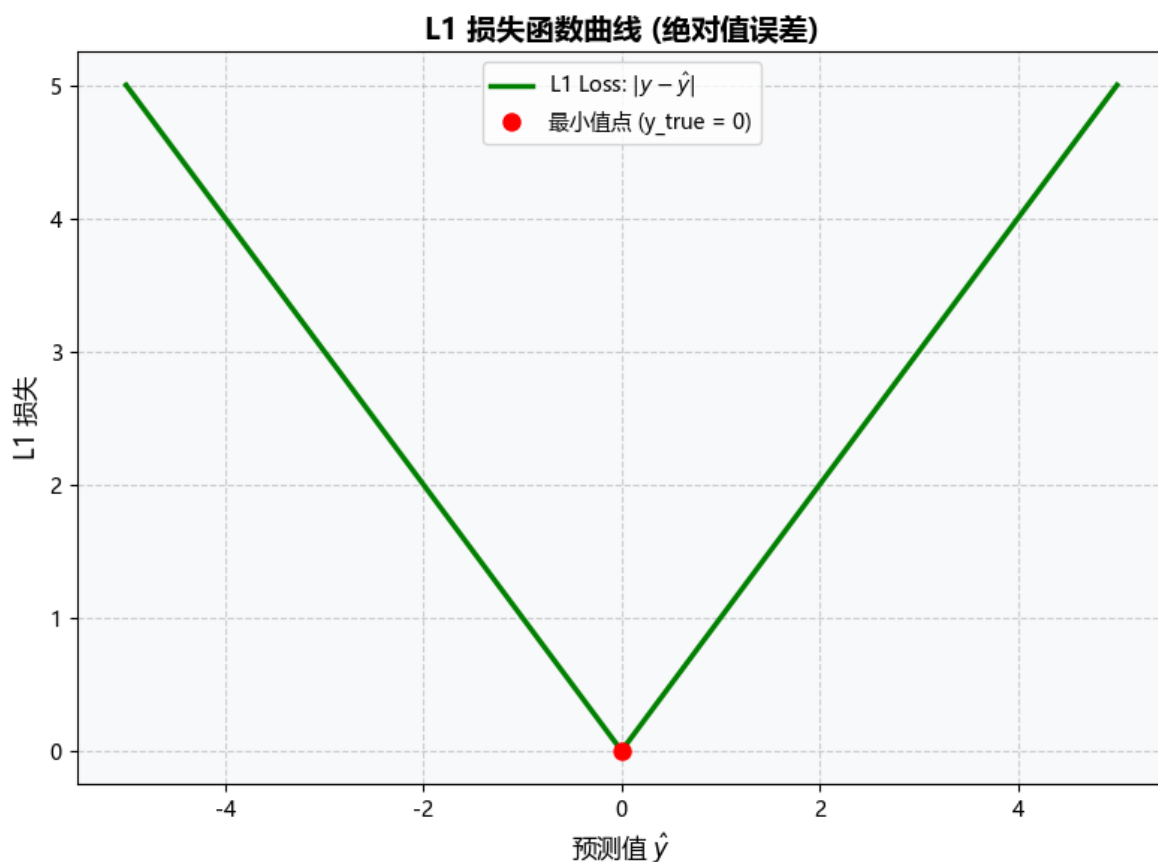
- 由于 L1 loss 具有稀疏性，为了惩罚较大的值，因此常常将其作为正则项添加到其他 loss 中作为约束。
- L1 loss 的最大问题是梯度在零点不平滑，导致会跳过极小值。
- 适用于回归问题中存在异常值或噪声数据时，可以减少对离群点的敏感性。

代码示例：

```
y_pred = torch.tensor([1.0, 1.0, 1.9], requires_grad=True)
y_true = torch.tensor([2.0, 2.0, 2.0])

criterion = nn.L1Loss()
loss = criterion(y_pred, y_true)
print(loss.item())    # (|1-2| + |1-2| + |1.9-2|)/3 = (1+1+0.1)/3 = 0.7
```

图例：



3.2 MSE 损失 (L2 Loss)

Mean Squared Loss / Quadratic Loss (MSE loss) 也被称为 L2 loss，或欧氏距离，它以误差的平方和的均值为距离。

公式：

$$\text{MSELoss}(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

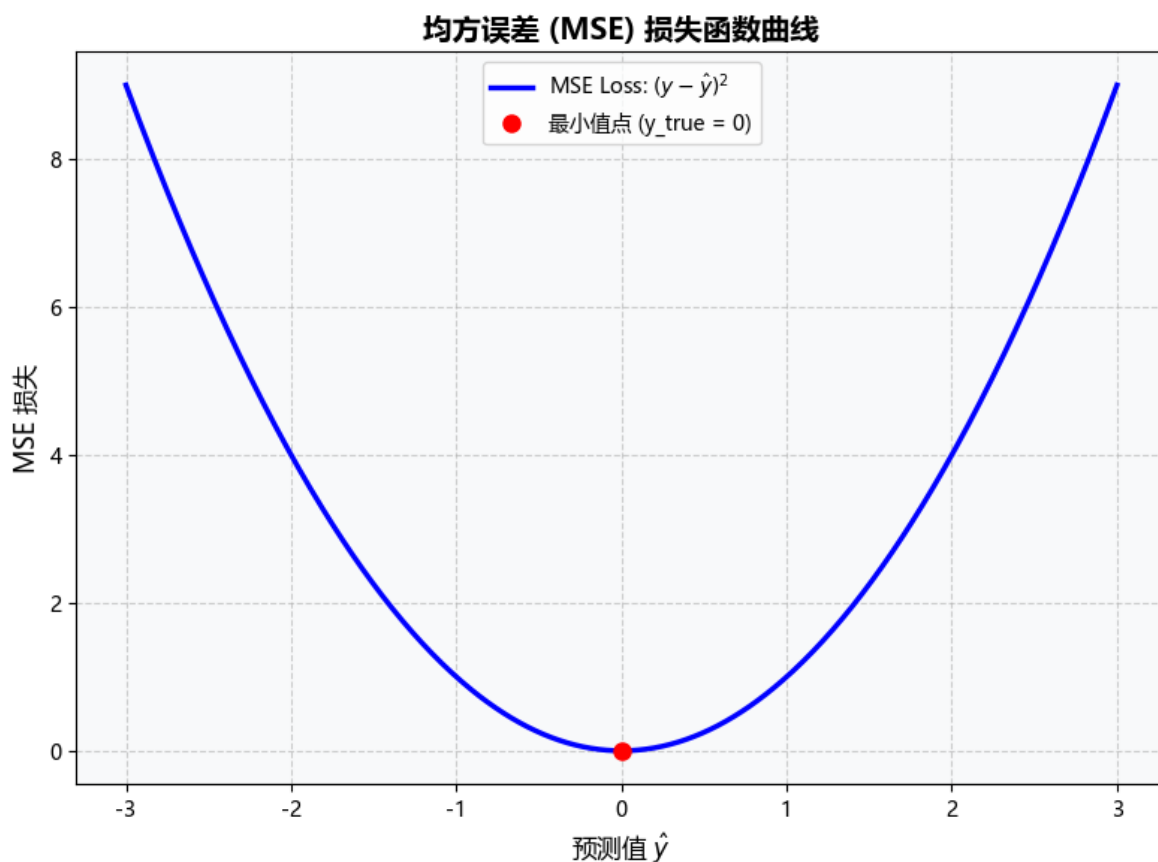
特点:

- L2 loss 也常常作为正则项，对于离群点 (outliers) 敏感，因为平方项会放大大误差。
- 当预测值与目标值相差很大时，梯度容易爆炸。梯度爆炸：网络层之间的梯度（值大于 1.0）重复相乘导致的指数级增长会产生梯度爆炸。
- 适用于大多数标准回归问题，如房价预测、温度预测等。

代码示例:

```
criterion = nn.MSELoss()  
loss = criterion(y_pred, y_true) # (1^2 + 1^2 + 0.1^2)/3 = 0.67  
print(loss.item())
```

图例:



- 3.3 Smooth L1 损失

Smooth L1 损失函数公式如下：

$$\text{SmoothL1 Loss}(x) = \begin{cases} 0.5x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise} \end{cases}$$

其中 $x = f(x) - y$ 为真实值和预测值的差值。

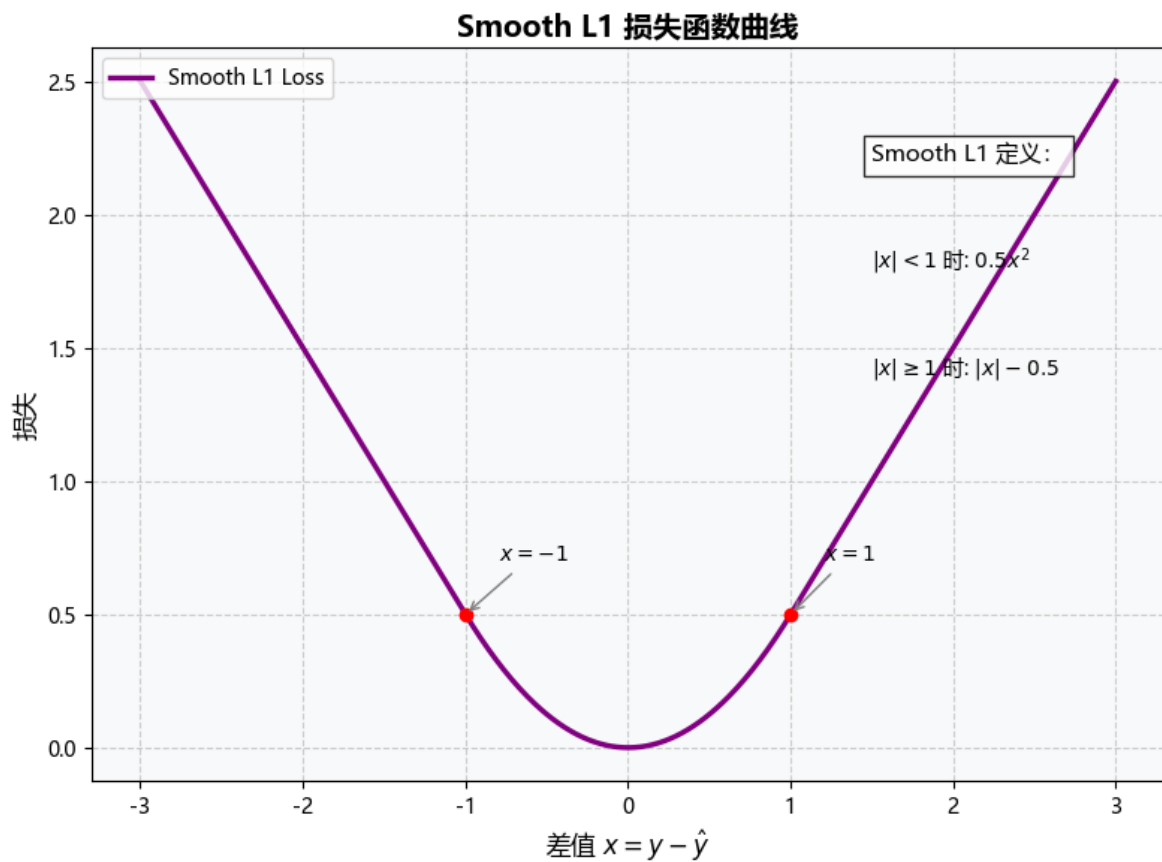
特点：

- 从函数图像可以看出，该函数实际上就是一个分段函数：在 $[-1,1]$ 之间实际上就是 L2 损失，这样解决了 L1 的不光滑问题；在 $[-1,1]$ 区间外，实际上就是 L1 损失，这样就解决了离群点梯度爆炸的问题。
- 对离群点更加鲁棒：当误差较大时，损失函数会线性增加（而不是像 MSE 那样平方增加），因此它对离群点的惩罚更小，避免了 MSE 对离群点过度敏感的问题。
- 计算梯度时更加平滑：与 MAE 相比，Smooth L1 在小误差时表现得像 MSE，避免了在训练过程中使用绝对误差而导致的梯度不连续问题。
- 常用于目标检测中的边界框回归（如 Faster R-CNN）。

代码示例：

```
criterion = nn.SmoothL1Loss()
loss = criterion(y_pred, y_true)
print(loss.item())
```

图例：



• 4. 损失函数选择总结

任务类型	推荐损失函数	PyTorch 实现	说明
多分类	交叉熵损失	<code>nn.CrossEntropyLoss</code>	输入为 logits, 标签为索引
二分类	带 logits 的 BCE	<code>nn.BCEWithLogitsLoss</code>	输入为 logits, 标签为 0/1
二分类 (概率)	BCE	<code>nn.BCELoss</code>	输入为概率 (需先 Sigmoid)
回归 (通用)	MSE Loss	<code>nn.MSELoss</code>	平滑, 对异常值敏感
回归 (稳健)	MAE Loss	<code>nn.L1 Loss</code>	对异常值鲁棒
回归 (平衡)	Smooth L1 Loss	<code>nn.SmoothL1 Loss</code>	结合 L1/L2 优点

其他常用损失:

- **KL 散度**: `nn.KLDivLoss` , 用于衡量两个概率分布的距离。
- **余弦嵌入损失**: `nn.CosineEmbeddingLoss` , 用于度量学习。
- **铰链损失**: `nn.HingeEmbeddingLoss` , 用于 SVM 等。

• 5. 损失函数与评估指标的区别

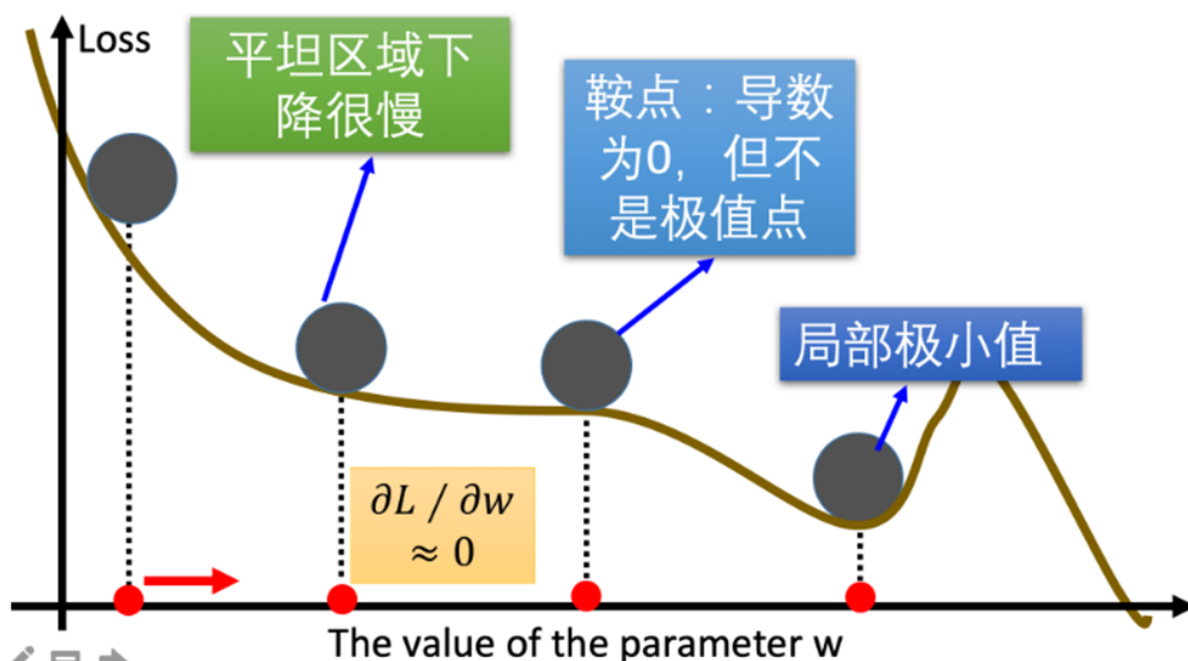
- **损失函数**: 用于**优化**, 必须可导, 通常在整个训练集上计算并反向传播。
- **评估指标** (如准确率、F1、AUC) : 用于**评估**模型性能, 不要求可导, 在验证/测试阶段使用。

例如, 在多分类任务中, 训练时使用交叉熵损失, 但监控的指标通常是准确率。

第五部分：网络优化方法

神经网络训练的核心是使用优化算法不断更新模型参数, 以最小化损失函数。传统的批量梯度下降 (BGD) 每次迭代使用全部数据计算梯度, 计算量大且容易陷入局部极小值; 随机梯度下降 (SGD) 每次使用一个样本, 波动大但收敛快; 小批量梯度下降 (Mini-batch SGD) 结合两者优点, 是最常用的基础优化器。然而, 标准 SGD 在遇到平缓区域、鞍点或局部最小值时可能收敛缓慢。为此, 研究者提出了多种改进的优化算法。

• 1. 优化算法面临的问题



- **平缓区域 (Plateau)** : 梯度值很小, 参数更新极慢。

- **鞍点 (Saddle Point)** : 梯度为零, 但既不是局部极小也不是极大, 参数无法更新。
- **局部最小值 (Local Minima)** : 梯度为零, 但不是全局最优。
- **震荡 (Oscillation)** : 由于 mini-batch 随机采样, 梯度方向可能在不同批次间剧烈变化, 导致收敛路径曲折。

• 2. 指数加权平均 (Exponential Weighted Moving Average, EWMA)

许多优化算法 (如 Momentum、RMSProp、Adam) 都依赖指数加权平均来平滑梯度或梯度平方。其公式为:

$$S_t = \beta S_{t-1} + (1 - \beta) Y_t$$

- S_t : 第 t 时刻的加权平均值。
- Y_t : 第 t 时刻的观测值 (如梯度)。
- β : 衰减率, 通常取 0.9 或 0.99, 值越大平均值越平滑。

作用: 赋予最近的数据更大的权重, 过去的数据权重指数衰减, 从而减少噪声, 使更新方向更稳定。

指数加权平均的原理: 我们最常见的算数平均指的是将所有数加起来除以数的个数, 每个数的权重是相同的。指数加权平均指的是给每个数赋予不同的权重求得平均数。移动平均数, 指的是计算最近邻的 N 个数来获得平均数。指数移动加权平均则是参考各数值, 并且各数值的权重都不同, 距离越远的数字对平均数计算的贡献就越小 (权重较小), 距离越近则对平均数的计算贡献就越大 (权重越大)。比如: 明天气温怎么样, 和昨天气温有很大关系, 而和一个月前的气温关系就小一些。

示例推导 (第 100 天的指数加权平均值) :

$$S_{100} = \beta S_{99} + (1 - \beta) Y_{100}$$

展开后各项系数随着距离指数衰减。

代码演示 (模拟 30 天气温的 EWMA) :

```
import torch
import matplotlib.pyplot as plt

torch.manual_seed(0)
temperature = torch.randn(30) * 10  # 随机气温

def exp_weighted_avg(data, beta=0.9):
    avg = []
```

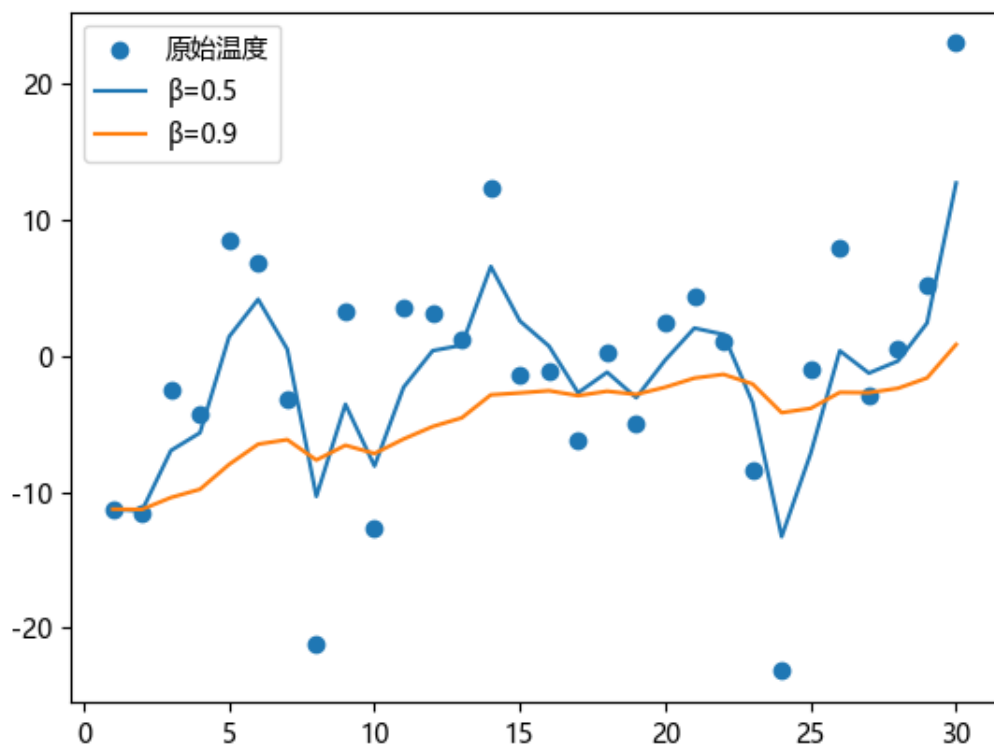
```

for i, temp in enumerate(data, 1):
    if i == 1:
        avg.append(temp)
    else:
        new_avg = avg[-1] * beta + (1 - beta) * temp
        avg.append(new_avg)
return avg

days = torch.arange(1, 31)
plt.scatter(days, temperature, label='原始温度')
plt.plot(days, exp_weighted_avg(temperature, beta=0.5), label='β=0.5')
plt.plot(days, exp_weighted_avg(temperature, beta=0.9), label='β=0.9')
plt.legend()
plt.show()

```

从图中可见， β 越大（如 0.9），曲线越平滑，波动越小。



• 3. 主流优化算法详解

- 3.1 Momentum（动量法）

核心思想：利用梯度的指数加权平均来加速收敛并抑制震荡。相当于模拟物理中的动量：当前更新方向不仅取决于当前梯度，还受之前梯度积累的方向影响。

更新公式：

$$v_t = \beta v_{t-1} + (1 - \beta)g_t$$

$$w_t = w_{t-1} - \eta v_t$$

- v_t ：当前时刻的动量（梯度加权平均）。
- g_t ：当前 mini-batch 的梯度。
- β ：动量系数，通常取 0.9。
- η ：学习率。

优点：

- 加速收敛，尤其在平缓区域或梯度方向一致的场景。
- 能够跨越鞍点和小的局部极小值（因为积累了历史梯度）。

对问题的改善：

- 当处于鞍点位置时，由于当前的梯度为 0，参数无法更新。但是 Momentum 动量梯度下降算法已经在先前积累了一些梯度值，很有可能使得跨过鞍点。
- 由于 mini-batch 普通的梯度下降算法，每次选取少数的样本梯度确定前进方向，可能会出现震荡，使得训练时间变长。Momentum 使用移动加权平均，平滑了梯度的变化，使得前进方向更加平缓，有利于加快训练过程。一定程度上有利于降低“峡谷”问题的影响。

PyTorch 实现：

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
```

示例（演示动量对参数更新的影响）：

```
w = torch.tensor([1.0], requires_grad=True)
loss = (w ** 2) / 2
optimizer = torch.optim.SGD([w], lr=0.01, momentum=0.9)
optimizer.zero_grad()
loss.backward()
optimizer.step()
print(f'梯度: {w.grad.item():.6f}, 更新后权重: {w.item():.6f}')
# 输出: 梯度: 1.000000, 更新后权重: 0.990000
# 第二次更新后权重会进一步减小: 0.971100
```

3.2 AdaGrad (自适应梯度算法)

核心思想：为每个参数分配不同的学习率，对频繁更新的参数使用较小学习率，对稀疏更新的参数使用较大学习率。通过累积历史梯度平方和来实现。

更新公式：

$$r_t = r_{t-1} + g_t \odot g_t$$

$$\Delta w_t = -\frac{\eta}{\sqrt{r_t + \epsilon}} \odot g_t$$

$$w_t = w_{t-1} + \Delta w_t$$

- r_t : 历史梯度平方的累积和。
- ϵ : 防止除零的小常数 (如 $1e-8$)。

计算步骤：

1. 初始化学习率 η 、初始化参数 w 、小常数 $\sigma = 1e-10$
2. 初始化梯度累计变量 $s = 0$
3. 从训练集中采样 m 个样本的小批量，计算梯度 g_t
4. 累积平方梯度: $s_t = s_{t-1} + g_t \odot g_t$
5. 学习率 η 的计算公式: $\eta = \frac{\eta}{\sqrt{s_t + \sigma}}$
6. 权重参数更新: $w_t = w_{t-1} - \frac{\eta}{\sqrt{s_t + \sigma}} * g_t$

优点：适合稀疏数据 (如 NLP 中的词嵌入) 和特征维度差异大的问题。

缺点：学习率单调递减，训练后期学习率过小，导致模型难以继续学习 (可能会使得学习率过早、过量的降低)。

PyTorch 实现：

```
optimizer = torch.optim.Adagrad(model.parameters(), lr=0.01)
```

- 3.3 RMSProp (均方根传播)

核心思想：针对 AdaGrad 学习率急剧下降的问题，使用指数加权平均代替累积平方和，从而保留近期梯度信息。

更新公式：

$$s_t = \beta s_{t-1} + (1 - \beta) g_t \odot g_t$$

$$w_t = w_{t-1} - \frac{\eta}{\sqrt{s_t + \epsilon}} \odot g_t$$

- β : 衰减率，通常取 0.9。

计算步骤：

1. 初始化学率 η 、初始化权重参数 w 、小常数 $\sigma = 1e - 10$
2. 初始化梯度累计变量 $s = 0$
3. 从训练集中采样 m 个样本的小批量，计算梯度 g_t
4. 使用指数加权平均累计历史梯度： $s_t = \beta s_{t-1} + (1 - \beta) g_t \odot g_t$
5. 学习率 η 的计算公式： $\eta = \frac{\eta}{\sqrt{s_t + \sigma}}$
6. 权重参数更新： $w_t = w_{t-1} - \frac{\eta}{\sqrt{s_t + \sigma}} * g_t$

优点：适用于非平稳目标（如 RNN），学习率不会单调递减到零。RMSProp 与 AdaGrad 最大的区别是对梯度的累积方式不同，对于每个梯度分量仍然使用不同的学习率。RMSProp 通过引入衰减系数 β ，控制历史梯度对历史梯度信息获取的多少，被证明在神经网络非凸条件下的优化更好，学习率衰减更加合理一些。

PyTorch 实现：

```
optimizer = torch.optim.RMSprop(model.parameters(), lr=0.01,
alpha=0.9)
```

- 3.4 Adam (自适应矩估计)

核心思想：结合 Momentum（一阶矩）和 RMSProp（二阶矩），同时对梯度的一阶矩和二阶矩进行指数加权平均，并加入偏差修正，使得算法在训练初期也能稳定。

更新公式（简化）：

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$w_t = w_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

- β_1, β_2 : 常用默认值 $\beta_1 = 0.9, \beta_2 = 0.999$ 。
- t : 时间步数。
- 偏差修正解决了初始时刻 m_t, v_t 偏向 0 的问题。

原理：Adam 是结合了 Momentum 和 RMSProp 优化算法的优点的自适应学习率算法。它计算了梯度的一阶矩（平均值）和二阶矩（梯度的方差）的自适应估计，从而动态调整学习率。

优点：

- 自适应学习率，对超参数敏感度较低。
- 训练快速、稳定，是当前最常用的优化器之一。

缺点：在某些情况下可能泛化性能不如 SGD + Momentum（尤其对于图像分类任务）。

PyTorch 实现：

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001, betas=(0.9, 0.999))
```

• 4. 优化器对比与选择建议

优化器	自适应学习率	适用场景	缺点
SGD + Momentum	否	大多数 CV 任务，需要精细调参	需要手动调整学习率
AdaGrad	是	稀疏数据（如 NLP）	学习率过早过小
RMSProp	是	RNN、强化学习	可能不稳定
Adam	是	大多数任务，默认首选	泛化性有时不如 SGD

优化器选择小结：

- 简单任务和较小的模型：SGD 或 Momentum

- 复杂任务或有大量数据：Adam 是最常用的选择，因其在大部分任务上都表现优秀
- 需要处理稀疏数据或文本数据：Adagrad 或 RMSProp

实践建议：

- **快速原型**：直接使用 Adam (lr=0.001) 。
- **图像分类**：尝试 SGD + Momentum (lr=0.01~0.1, momentum=0.9) ，配合学习率衰减。
- **NLP / 稀疏特征**：Adam 或 RMSProp。
- **需要收敛到更优极值**：SGD + Momentum 往往比 Adam 泛化更好。

• 5. 学习率衰减 (Learning Rate Scheduling)

为什么要进行学习率优化？在训练神经网络时，一般情况下学习率都会随着训练而变化。这主要是由于，在神经网络训练的后期，如果学习率过高，会造成 loss 的振荡，但是如果学习率减小的过慢，又会造成收敛变慢的情况。

下面通过代码来理解学习率设置不同对网络训练的影响：采用较小的学习率，梯度下降的速度慢；采用较大的学习率，梯度下降太快越过了最小值点，导致不收敛，甚至震荡（梯度爆炸）。

```
import torch
import matplotlib.pyplot as plt

def func(x_t):
    return torch.pow(2 * x_t, 2) # y = 4 x^2

def dm_01(lr=0.1):
    x = torch.tensor([2.], requires_grad=True)
    iter_rec, loss_rec = [], []
    for i in range(4):
        y = func(x)
        y.backward()
        print(f"Iter {i}: X={x.item():.6 f}, X.grad={x.grad.item():.6 f}, loss={y.item():.6 f}")
        x.data.sub_(lr * x.grad)
        x.grad.zero_()
        iter_rec.append(i)
        loss_rec.append(y.item())
    # 绘图代码省略...
```


训练过程中逐步降低学习率有助于在接近最优解时进行精细调整，避免震荡。PyTorch 的 `torch.optim.lr_scheduler` 提供了多种策略。

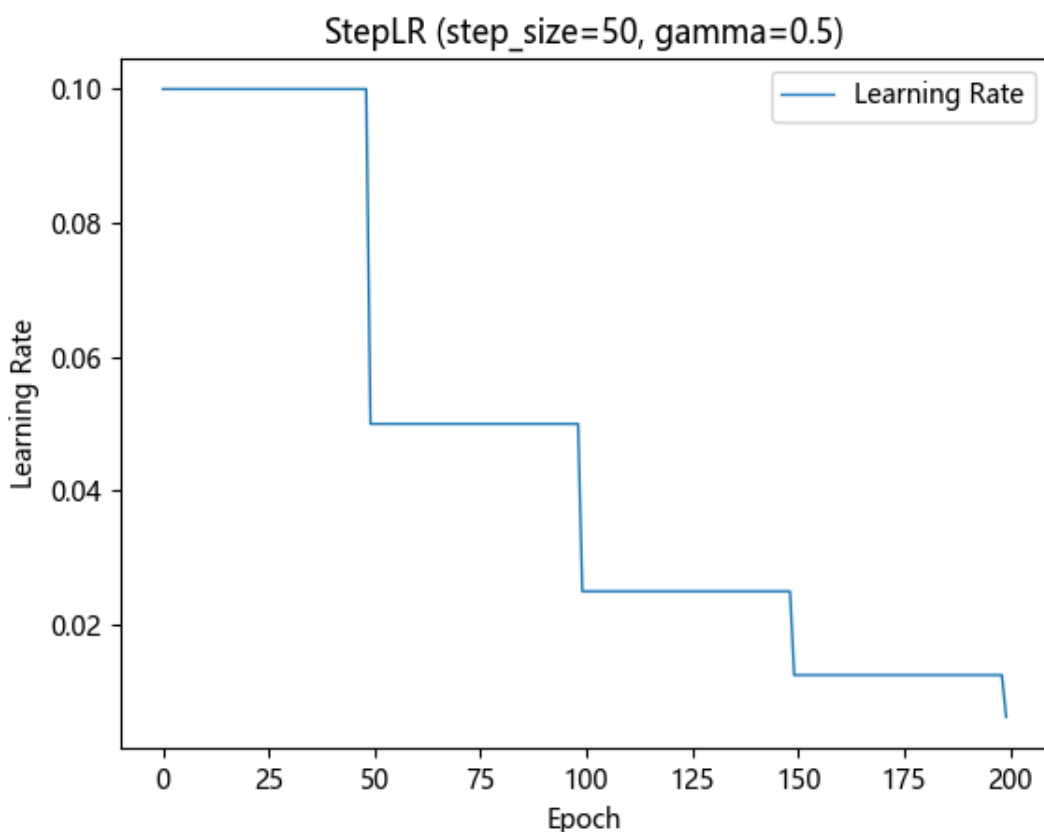
- 5.1 等间隔衰减 (StepLR)

每隔 `step_size` 个 epoch，学习率乘以 `gamma`。

```
scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=50, gamma=0.5)
```

在每个 epoch 结束后调用 `scheduler.step()`。

图例：

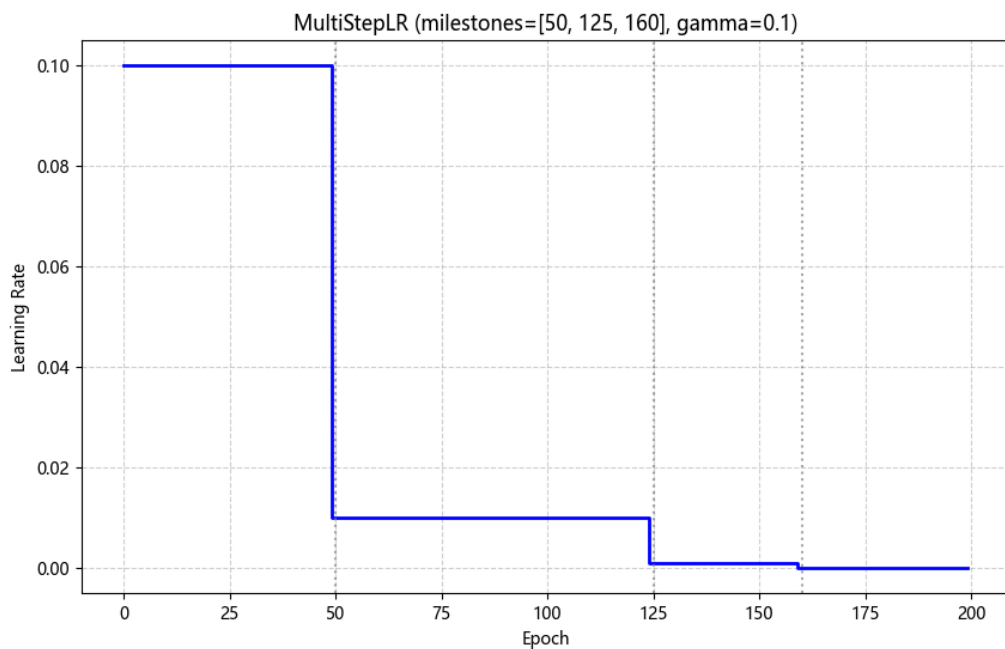


- 5.2 指定间隔衰减 (MultiStepLR)

在指定的 epoch 节点（如 [50, 125, 160]）对学习率乘以 `gamma`。

```
scheduler = torch.optim.lr_scheduler.MultiStepLR(optimizer, milestones=[50, 125, 160], gamma=0.5)
```

图例：

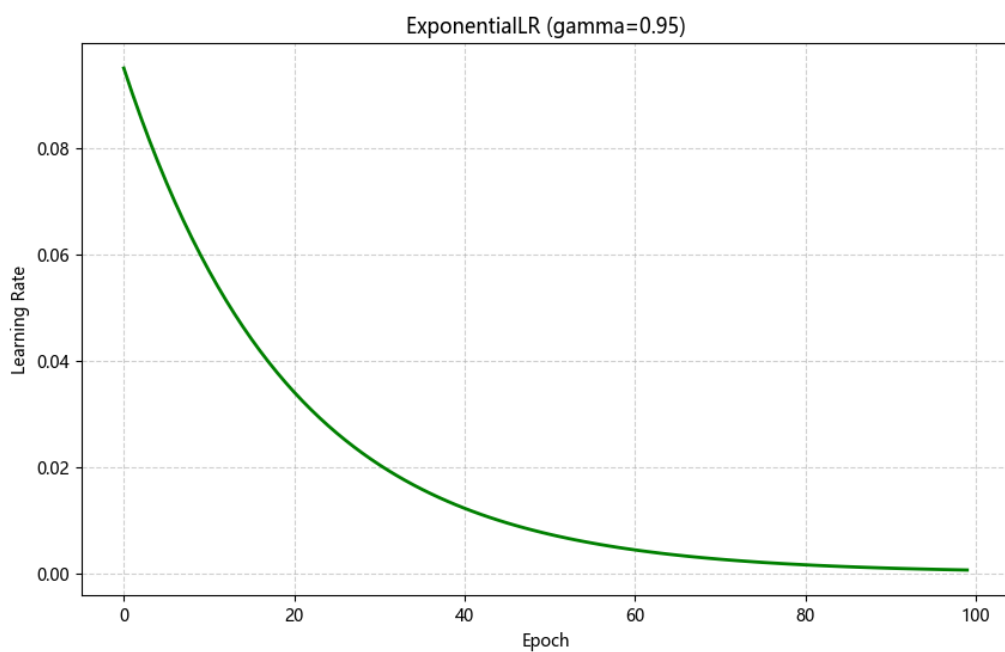


- 5.3 指数衰减 (ExponentialLR)

每个 epoch 学习率乘以 $\text{gamma}^{\text{epoch}}$ 。

```
scheduler = torch.optim.lr_scheduler.ExponentialLR(optimizer,  
gamma=0.95)
```

图例：



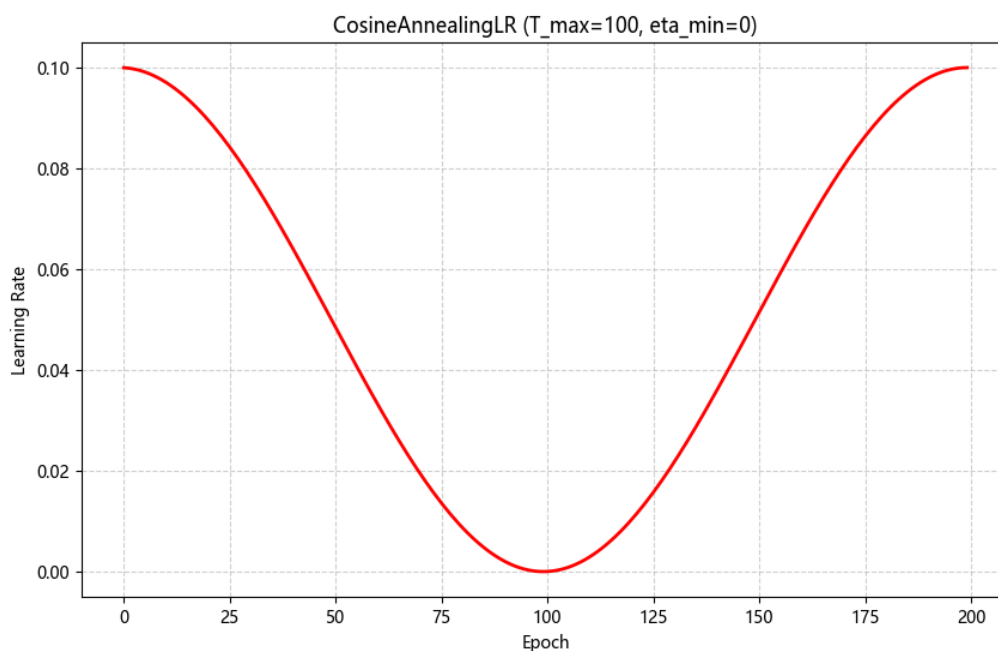
- 5.4 余弦退火 (CosineAnnealingLR)

学习率按照余弦函数周期变化，公式：

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos\left(\frac{T_{cur}}{T_{max}}\pi\right))$$

```
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer,  
T_max=50)
```

图例：



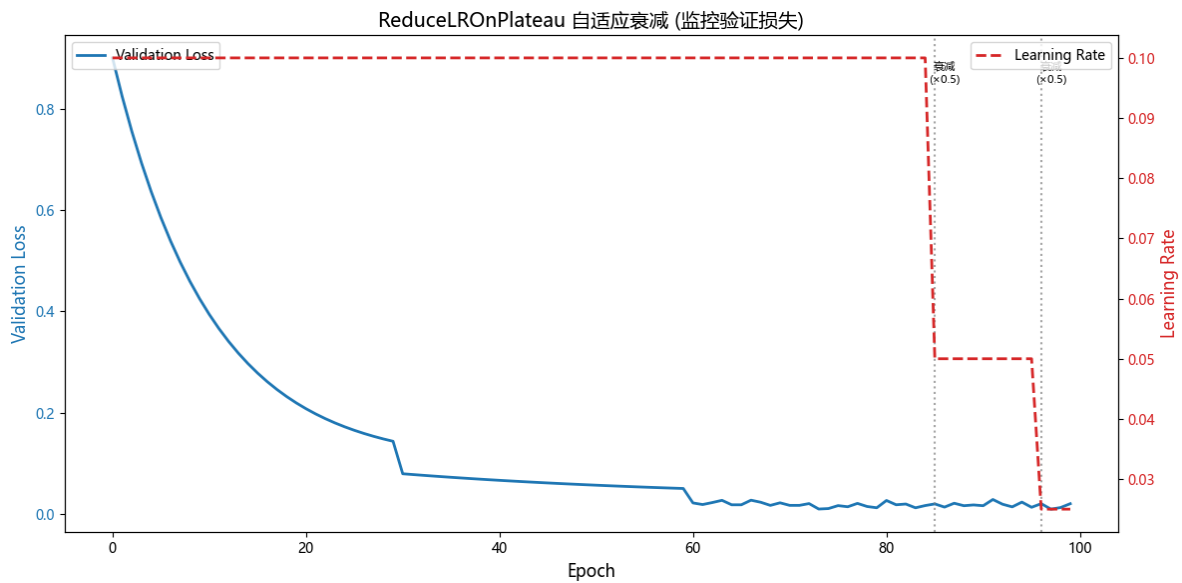
- 5.5 自适应衰减 (ReduceLROnPlateau)

当验证集指标停止提升时，降低学习率。

```
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer,  
mode='min', patience=10, factor=0.1)
```

需要将验证损失传入 `scheduler.step(val_loss)`。

图例：



• 6. 完整训练循环模板（含优化器和学习率调度）

```
import torch.optim as optim
from torch.optim.lr_scheduler import StepLR

model = ... # 模型
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
scheduler = StepLR(optimizer, step_size=30, gamma=0.1)

for epoch in range(num_epochs):
    model.train()
    for inputs, labels in train_loader:
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

    scheduler.step() # 更新学习率
    current_lr = optimizer.param_groups[0]['lr']
    print(f"Epoch {epoch+1}, LR: {current_lr:.6 f}")
```

7. 学习率衰减方法总结

方法	衰减方式	实现难度	适用场景
等间隔学习率衰减 (StepLR)	固定步长衰减	简单易实现	大型数据集、较为简单的任务
指定间隔学习率衰减 (MultiStepLR)	指定步长衰减	相对简单，容易调整	对训练平稳性要求较高的任务
指数学习率衰减 (ExponentialLR)	平滑指数衰减	较复杂，需额外历史计算	高精度训练，避免过快收敛

第六部分：正则化方法

神经网络强大的表达能力是一把双刃剑：当模型参数远多于训练样本时，很容易发生**过拟合**（overfitting），即模型在训练集上表现极好，但在未见过的测试集上泛化能力差。**正则化**（Regularization）是一类用于缓解过拟合、提高模型泛化能力的策略。深度学习中常用的正则化方法包括范数惩罚（权重衰减）、Dropout、批量归一化（Batch Normalization）等。

1. 正则化的作用

- 降低模型复杂度**：限制模型参数的自由度，防止其过度记忆训练数据中的噪声。
- 提高泛化能力**：使模型在测试集上也能获得稳定且较好的表现。
- 平滑决策边界**：正则化倾向于让模型学习更平滑、更简单的函数。

2. 范数惩罚（权重衰减）

范数惩罚通过在损失函数中增加参数 W 的范数项来约束权重大小。最常用的是 **L2 正则化**（权重衰减，Weight Decay）。

L2 正则化公式：

$$L_{\text{total}}(W) = L_{\text{original}}(W) + \frac{\lambda}{2} \|W\|_2^2$$

其中 $\|W\|_2^2 = \sum_i w_i^2$ ， λ 为正则化系数（通常为 0.001 ~ 0.0001）。

效果：使权重趋向于较小的值，但不为零。小权重意味着模型对输入的微小变化不敏感，从而提高稳定性。

L1 正则化 (Lasso)：

$$L_{\text{total}}(W) = L_{\text{original}}(W) + \lambda \|W\|_1$$

L1 正则化会使部分权重变为 0，产生稀疏解，可用于特征选择。

PyTorch 中的权重衰减：

在优化器中直接设置 `weight_decay` 参数即可，等价于 L2 正则化。

```
# SGD 中设置 weight_decay
optimizer = torch.optim.SGD(model.parameters(), lr=0.01,
weight_decay=1e-4)

# Adam 中也支持
optimizer = torch.optim.Adam(model.parameters(), lr=0.001,
weight_decay=1e-4)
```

注意事项：

- 权重衰减通常不对偏置项使用，因为偏置对模型复杂度影响较小。
- 正则化系数 λ 需要根据验证集调优，过大会导致欠拟合。

• 3. Dropout (随机失活)

Dropout (随机失活) 是由 Hinton 等人提出的一种简单而有效的正则化方法，特别适用于全连接层。

- 3.1 Dropout 的原理

- **训练阶段：**每个神经元以超参数 p (丢弃概率，通常称为 `dropout rate`) 的概率被**临时丢弃** (即其输出置为 0)。被丢弃的神经元在该次前向传播和反向传播中不参与计算。未被丢弃的神经元的输出会乘以缩放因子 $\frac{1}{1-p}$ ，以保持整体激活的期望不变。训练过程可以认为是对完整的神经网络的一些子集进行训练，每次基于输入数据只更新子网络的参数。
- **测试阶段：**随机失活不起作用，所有神经元都参与计算，但输出不再缩放 (相当于使用了完整的网络)。

缩放的必要性：在训练阶段，将参与计算的神经元的输出除以 $(1 - p)$ ，经过 Dropout 后的期望输出变为 $E[x_{\text{dropout}}] = [(1 - p) \cdot x] / (1 - p) = x$ ，与测试阶段的期望输出一致。

代码示例：

```
import torch
import torch.nn as nn

dropout = nn.Dropout(p=0.4)  # p 为丢弃概率
inputs = torch.randint(0, 10, (1, 4)).float()
linear = nn.Linear(4, 5)
x = linear(inputs)
x = torch.relu(x)
print("未失活输出:", x)

x_drop = dropout(x)
print("失活后输出:", x_drop)
# 输出中有一些元素变为 0，未变为 0 的会放大  $1/(1-0.4) \approx 1.667$  倍
```

- 3.2 Dropout 的使用建议

- **只在训练时启用 Dropout：**PyTorch 的 `nn.Dropout` 会自动根据 `model.train()` 和 `model.eval()` 状态切换行为。在 `eval()` 模式下 Dropout 不生效。
 - **适用位置：**通常放在全连接层之后、激活函数之前或之后。对于卷积层，有时也使用 Dropout 2 d（随机丢弃整个通道）。
 - **常用丢弃率：**
 - 输入层：0.2 左右
 - 隐藏层：0.3 ~ 0.5
 - 输出层：通常不使用 Dropout
 - **深层网络：**越接近输出层的隐藏层，丢弃率可以适当降低。
 - 对于较小的模型或较复杂的任务，丢弃率可以选择 0.3 或更小；对于非常深的网络，较大的丢弃率（如 0.5 或 0.6）可能会有效防止过拟合。
-

- 3.3 Dropout 的缺点

- 训练时间变长（因为需要训练更多子网络）。
 - 在小型数据集或简单模型上可能效果不明显。
-

• 4. 批量归一化 (Batch Normalization, BN)

批量归一化由 Google 在 2015 年提出，最初用于解决深层网络训练中的**内部协变量偏移** (Internal Covariate Shift) 问题，即每层输入分布随着前面层参数变化而不断改变，导致训练不稳定。BN 通过对每个 mini-batch 的数据进行归一化，再引入可学习的缩放和平移参数，使网络训练更快速、更稳定。

- 4.1 BN 的计算过程

对于一个 batch 中某个特定通道的输入 x ，BN 层执行以下操作：

1. **计算 mini-batch 的均值**： $\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$
2. **计算 mini-batch 的方差**： $\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$
3. **归一化**： $\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$ (ϵ 为防止除零的小常数，如 10^{-5})
4. **缩放和平移**： $y_i = \gamma \hat{x}_i + \beta$

其中 γ 和 β 是**可学习的参数**（与权重一样通过反向传播更新），恢复网络一定的表达能力，因为纯粹的归一化可能破坏原始特征分布。

- 4.2 BN 的优点

- **加速收敛**：可以使用更大的学习率，减少对参数初始化的敏感度。
 - **缓解过拟合**：具有轻微的正则化效果，因为每个 batch 的均值和方差有微小噪声。
 - **允许更高的学习率**：减少了梯度爆炸/消失的风险。
 - **减少了对 Dropout 的依赖**：在一些网络中，BN 可以部分替代 Dropout。
 - **提升泛化能力**：由于其正则化效果，批量归一化能帮助网络在测试集上取得更好的性能。
-

- 4.3 BN 在卷积网络 and 全连接网络中的区别

- **全连接层**：对每个神经元分别计算均值和方差，即每个特征维度独立进行 BN。输入形状 (N, D) ，输出形状 (N, D) 。PyTorch 中使用 `nn.BatchNorm1d`。
- **卷积层**：BN 通常放在卷积层之后、激活函数之前。计算时对每个通道 (channel) 独立进行，即同一个通道内的所有像素共享相同的均值和方差。输入形状 (N, C, H, W) ，输出形状 (N, C, H, W) 。PyTorch 中使用 `nn.BatchNorm2d`。

PyTorch 提供了对应的 BN 层：

- `nn.BatchNorm1d`：用于 2D 输入 (N, C) 或 3D 序列 (N, C, L)
- `nn.BatchNorm2d`：用于 4D 图像 (N, C, H, W)
- `nn.BatchNorm3d`：用于 5D 视频/体积数据 (N, C, D, H, W)

- 4.4 代码示例

```
import torch.nn as nn

# 假设输入形状 (batch, channels, height, width)
bn = nn.BatchNorm2d(num_features=2) # 通道数为 2
input = torch.randn(1, 2, 3, 4)
output = bn(input)

print("BN 可学习的权重  $\gamma$ :", bn.weight) # 初始为 1
print("BN 可学习的偏置  $\beta$ :", bn.bias) # 初始为 0
print("输出形状:", output.shape)
```

训练 vs 测试：

- 训练时：BN 使用当前 batch 的均值和方差。
- 测试时：BN 使用训练过程中**通过滑动平均**累积的全局均值和方差。PyTorch 会通过 `model.eval()` 自动切换。

- 4.5 BN 的注意事项

- **batch size 不宜过小**：一般要求 $\text{batch size} \geq 16$ ，否则均值和方差估计不稳定。对于极小 batch (如 1 ~ 4)，可改用 Layer Normalization 或 Instance Normalization。
- **RNN 中谨慎使用**：RNN 的不同时间步共享 BN 层，效果不如 Layer Normalization (如 `nn.LayerNorm`)。

- **位置**：通常放在线性层/卷积层之后、激活函数之前，即 `Conv -> BN -> ReLU`。

5. 其他正则化方法

方法	说明
早停 (Early Stopping)	当验证集损失不再下降时提前终止训练，防止过拟合。
数据增强 (Data Augmentation)	对输入数据施加随机变换（旋转、翻转、裁剪、加噪等），增加训练样本多样性。
标签平滑 (Label Smoothing)	将 one-hot 标签的 1 替换为 $1 - \epsilon$ ，0 替换为 $\epsilon / (K - 1)$ ，防止模型过于自信。
Layer Normalization	对每个样本的所有特征维度进行归一化，适用于 RNN 和 Transformer。
Instance Normalization	对每个样本的每个通道独立归一化，用于风格迁移。

6. 正则化方法选择建议

场景	推荐正则化策略
小型全连接网络	Dropout + 权重衰减
图像分类 CNN	Batch Normalization + 权重衰减（可减少 Dropout）
大型模型 + 小数据集	Dropout + 数据增强 + 早停
Transformer / NLP	Layer Normalization + 标签平滑
生成对抗网络 (GAN)	通常使用权重衰减，BN 在某些 GAN 变体中需要谨慎使用

第七部分：完整案例——手机价格分类（模型构建、训练、调优）

本案例基于前六部分的知识，构建一个全连接神经网络，解决**手机价格区间分类**问题。数据集包含二手手机的多项性能指标（如 RAM、电池容量、摄像头像素等），目标是将手机划分为 4 个价格区间（0、1、2、3，数值越大价格越高）。这是一个典型的多分类任务。

我们将按照标准的机器学习/深度学习流程进行：**数据准备** → **模型设计** → **训练与验证** → **评估与调优**。

• 1. 需求分析

小明创办了一家手机公司，他不知道如何估算手机产品的价格。为了解决这个问题，他收集了多家公司的手机销售数据。该数据为二手手机的各个性能的数据，最后根据这些性能得到 4 个价格区间，作为这些二手手机售出的价格区间。主要包括：

- battery_power：电池一次可储存的总能量，单位为毫安时
- blue：是否有蓝牙
- clock_speed：微处理器执行指令的速度
- dual_sim：是否支持双卡
- fc：前置摄像头百万像素
- four_g：是否有 4 G
- int_memory：内存（GB）
- m_dep：移动深度（cm）
- mobile_wt：手机重量
- n_cores：处理器内核数
- pc：主摄像头百万像素
- px_height：像素分辨率高度
- px_width：像素分辨率宽度
- ram：随机存取存储器（兆字节）
- sc_h：手机屏幕高度（cm）
- sc_w：手机屏幕宽度（cm）
- talk_time：一次电池充电持续时间最长的时间
- three_g：是否有 3 G
- touch_screen：是否有触控屏

- wifi: 是否能连 wifi
- price_range: 价格区间 (0, 1, 2, 3)

我们需要帮助小明找出手机的功能 (例如: RAM 等) 与其售价之间的某种关系。我们可以使用机器学习的方法来解决这个问题, 也可以构建一个全连接的网络。需要注意的是: 在这个问题中, 我们不需要预测实际价格, 而是一个价格范围, 它的范围使用 0、1、2、3 来表示, 所以该问题也是一个分类问题。

• 2. 构建数据集

数据共有 2000 条, 其中 1600 条数据作为训练集, 400 条数据用作测试集。我们使用 sklearn 的数据集划分工作来完成。并使用 PyTorch 的 TensorDataset 来将数据集构建为 Dataset 对象, 方便构造数据集加载对象。

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import pandas as pd
import numpy as np
import time

# 设置随机种子保证可复现
torch.manual_seed(42)
np.random.seed(42)

def create_dataset(csv_path='data/手机价格预测.csv', test_size=0.2,
random_state=88):
    """
    加载数据, 划分训练/验证集, 并进行标准化
    返回: 训练集 Dataset 对象、验证集 Dataset 对象、输入维度、类别数
    """

    # 1. 读取数据
    data = pd.read_csv(csv_path)
    X = data.iloc[:, :-1].values.astype(np.float32)
    y = data.iloc[:, -1].values.astype(np.int 64)

    # 2. 划分训练集和验证集 (80% 训练, 20% 验证)
    X_train, X_valid, y_train, y_valid = train_test_split(
        X, y, test_size=test_size, random_state=random_state,
        stratify=y # 分层采样保证类别分布一致
    )
```

```
# 3. 标准化（重要！可显著提升收敛速度和稳定性）
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_valid = scaler.transform(X_valid)

# 4. 转换为 PyTorch 张量
train_dataset = TensorDataset(torch.from_numpy(X_train),
                                torch.from_numpy(y_train))
valid_dataset = TensorDataset(torch.from_numpy(X_valid),
                                torch.from_numpy(y_valid))

input_dim = X.shape[1]          # 特征数量（本例为 20）
num_classes = len(np.unique(y)) # 类别数量（本例为 4）

return train_dataset, valid_dataset, input_dim, num_classes

if __name__ == '__main__':
    train_dataset, valid_dataset, input_dim, class_num =
    create_dataset()
    print("输入特征数:", input_dim)    # 20
    print("分类个数:", class_num)      # 4
```

• 3. 构建分类网络模型

构建全连接神经网络来进行手机价格分类，该网络主要由三个线性层来构建，使用 ReLU 激活函数。网络共有 3 个全连接层，具体信息如下：

- 第一层：输入为维度为 20，输出维度为：128
- 第二层：输入为维度为 128，输出维度为：256
- 第三层：输入为维度为 256，输出维度为：4

在进阶版本中，我们还可以增加网络深度（例如 4 层）并加入 BatchNorm 和 Dropout。

- 3.1 基础版本模型

```
class PhonePriceModel(nn.Module):
    def __init__(self, input_dim, output_dim):
        super(PhonePriceModel, self).__init__()
        self.linear 1 = nn.Linear(input_dim, 128)
        self.linear 2 = nn.Linear(128, 256)
        self.linear 3 = nn.Linear(256, output_dim)

    def forward(self, x):
        x = torch.relu(self.linear 1(x))
        x = torch.relu(self.linear 2(x))
        output = self.linear 3(x)  # 后续 CrossEntropyLoss 中包含
softmax
        return output
```

- 3.2 进阶版本模型 (带 BatchNorm、Dropout、Kaiming 初始化)

```
class PhonePriceModelAdvanced(nn.Module):
    def __init__(self, input_dim, output_dim, hidden_dims=[128, 256],
dropout_rate=0.3):
        super(PhonePriceModelAdvanced, self).__init__()

        self.fc 1 = nn.Linear(input_dim, hidden_dims[0])
        self.bn 1 = nn.BatchNorm 1 d(hidden_dims[0])
        self.dropout 1 = nn.Dropout(dropout_rate)

        self.fc 2 = nn.Linear(hidden_dims[0], hidden_dims[1])
        self.bn 2 = nn.BatchNorm 1 d(hidden_dims[1])
        self.dropout 2 = nn.Dropout(dropout_rate)

        self.out = nn.Linear(hidden_dims[1], output_dim)

        self._initialize_weights()

    def _initialize_weights(self):
        for m in self.modules():
            if isinstance(m, nn.Linear):
                nn.init.kaiming_uniform_(m.weight, mode='fan_in',
nonlinearity='relu')
                if m.bias is not None:
                    nn.init.constant_(m.bias, 0)

    def forward(self, x):
        x = self.fc 1(x)
```

```
x = self.bn 1(x)
x = torch.relu(x)
x = self.dropout 1(x)

x = self.fc 2(x)
x = self.bn 2(x)
x = torch.relu(x)
x = self.dropout 2(x)

x = self.out(x)
return x
```

• 4. 模型训练与验证

- 4.1 训练函数（基础版）

```
def train(train_dataset, input_dim, class_num):
    torch.manual_seed(0)
    dataloader = DataLoader(train_dataset, shuffle=True, batch_size=8)
    model = PhonePriceModel(input_dim, class_num)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.SGD(model.parameters(), lr=1e-3)
    num_epoch = 50

    for epoch_idx in range(num_epoch):
        start = time.time()
        total_loss = 0.0
        total_num = 0
        for x, y in dataloader:
            output = model(x)
            loss = criterion(output, y)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            total_num += len(y)
            total_loss += loss.item() * len(y)
        print('epoch: %4s loss: %.2f, time: %.2fs' % (epoch_idx +
            1, total_loss / total_num, time.time() - start))

    torch.save(model.state_dict(), 'model/phone.pth')
```

- 4.2 训练函数（进阶版，带早停和学习率调度）

```
def train_model(model, train_dataset, valid_dataset,
                batch_size=64, lr=0.001, weight_decay=1e-4,
                num_epochs=100, patience=10):
    train_loader = DataLoader(train_dataset, batch_size=batch_size,
                              shuffle=True)
    valid_loader = DataLoader(valid_dataset, batch_size=batch_size,
                              shuffle=False)

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr,
                             weight_decay=weight_decay)
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer,
                                                       mode='min', factor=0.5, patience=5, verbose=True)

    best_valid_acc = 0.0
    best_epoch = 0
    epochs_no_improve = 0

    train_losses = []
    valid_accs = []

    for epoch in range(1, num_epochs + 1):
        # 训练阶段
        model.train()
        total_loss = 0.0
        for X_batch, y_batch in train_loader:
            optimizer.zero_grad()
            outputs = model(X_batch)
            loss = criterion(outputs, y_batch)
            loss.backward()
            optimizer.step()
            total_loss += loss.item() * X_batch.size(0)
        avg_train_loss = total_loss / len(train_dataset)
        train_losses.append(avg_train_loss)

        # 验证阶段
        model.eval()
        correct = 0
        total = 0
        with torch.no_grad():
            for X_batch, y_batch in valid_loader:
                outputs = model(X_batch)
                _, preds = torch.max(outputs, 1)
                correct += (preds == y_batch).sum().item()
                total += y_batch.size(0)
```



```

        valid_acc = correct / total
        valid_accs.append(valid_acc)

    scheduler.step(avg_train_loss)

    if epoch % 10 == 0 or epoch == 1:
        print(f"Epoch {epoch:3d} | Train Loss: {avg_train_loss:.4 f} | Valid Acc: {valid_acc:.4 f}")

    # 早停与保存最佳模型
    if valid_acc > best_valid_acc:
        best_valid_acc = valid_acc
        best_epoch = epoch
        torch.save(model.state_dict(), 'best_phone_model.pth')
        epochs_no_improve = 0
    else:
        epochs_no_improve += 1
        if epochs_no_improve >= patience:
            print(f"Early stopping at epoch {epoch}, best valid acc: {best_valid_acc:.4 f} at epoch {best_epoch}")
            break

    model.load_state_dict(torch.load('best_phone_model.pth'))
    return model, best_valid_acc, train_losses, valid_accs

```

4.3 评估函数

```

def test(valid_dataset, input_dim, class_num,
        model_path='model/phone.pth'):
    model = PhonePriceModel(input_dim, class_num)
    model.load_state_dict(torch.load(model_path))
    dataloader = DataLoader(valid_dataset, batch_size=8,
        shuffle=False)
    correct = 0
    for x, y in dataloader:
        model.eval()
        output = model(x)
        y_pred = torch.argmax(output, dim=1)
        correct += (y_pred == y).sum()
    print('Acc: %.5 f' % (correct.item() / len(valid_dataset)))

```

• 5. 模型预测与评估

运行训练和评估：

```
if __name__ == '__main__':  
    train_dataset, valid_dataset, input_dim, class_num =  
    create_dataset()  
    # 基础版本训练  
    train(train_dataset, input_dim, class_num)  
    test(valid_dataset, input_dim, class_num)  
    # 输出 Acc: 0.64250 (基线)
```

• 6. 网络性能优化（调优建议）

我们前面的网络模型在测试集的准确率为：0.64250，我们可以通过以下方面进行调优：

1. 对输入数据进行标准化（已在数据加载中实现）
2. 调整优化方法：由 SGD 调整为 Adam
3. 调整学习率：由 $1 \text{ e-}3$ 调整为 $1 \text{ e-}4$
4. 增加批量归一化层
5. 增加网络深度，即增加网络参数量
6. 增加训练轮数
7. 调整 Dropout 比率
8. 权重衰减
9. 学习率调度策略

下面给出一个优化后的完整流程示例（包含数据标准化、更深网络、Adam、早停等）：

```
# 优化后的版本（在 create_dataset 中已经包含标准化，模型使用
PhonePriceModelAdvanced）
if __name__ == '__main__':
    train_dataset, valid_dataset, input_dim, class_num =
create_dataset()
    model = PhonePriceModelAdvanced(input_dim, class_num, hidden_dims=
[128, 256, 512, 128], dropout_rate=0.3)
    trained_model, best_acc, losses, accs = train_model(
        model, train_dataset, valid_dataset,
        batch_size=64, lr=1 e-4, weight_decay=1 e-4,
        num_epochs=100, patience=10
    )
    print(f"最佳验证准确率: {best_acc:.4 f}")
# 绘制训练曲线和混淆矩阵等（可参考之前的绘图代码）
```

• 7. 调优具体步骤

1. **固定随机种子，建立基线。**
2. **先调整数据预处理**（标准化、特征选择）。标准化可以显著提升收敛速度和稳定性。
3. **调整模型结构**（隐藏层数和宽度）。增加深度（如从 3 层增加到 5 层）有时能提升表达能力，但需注意过拟合。
4. **调整正则化强度**（Dropout rate、weight_decay）。Dropout 一般从 0.3 开始，根据验证集表现增减。
5. **调整优化器及学习率**：Adam 默认学习率 0.001 通常不错，但可以尝试 0.0001 或 0.0005。对于 SGD，学习率通常需要更大（0.01~0.1）。
6. **学习率调度**：使用 ReduceLROnPlateau 或余弦退火。
7. **尝试集成方法**（训练多个模型投票、SWA）。

每次只改变一个变量，记录验证集准确率变化。

• 8. 项目总结

通过本案例，我们实践了：

- 使用 PyTorch 构建全连接神经网络。
- 数据加载、标准化和 Dataset/DataLoader 的使用。
- 自定义网络类（继承 `nn.Module`）并应用 BatchNorm、Dropout、Kaiming 初始化。

- 训练循环、验证、早停与学习率调度。
- 模型保存与评估（混淆矩阵、分类报告）。
- 调优思路与超参数选择。

完整的代码结构可以作为一个模板，适用于大多数表格数据的分类/回归任务。

神经网络知识总结

本笔记共七部分，系统覆盖了：

1. **神经网络概述与结构**：神经元、全连接网络、内部状态值、激活值、参数量计算。
2. **激活函数**：Sigmoid、Tanh、ReLU、Softmax 的原理、优缺点、代码示例和选择指南。
3. **参数初始化**：Xavier、Kaiming 等初始化方法的原理、适用场景和 PyTorch 实现。
4. **损失函数**：分类（交叉熵）、回归（MAE、MSE、Smooth L1）的公式和代码。
5. **优化方法**：SGD、Momentum、AdaGrad、RMSProp、Adam 的原理、对比和学习率调度。
6. **正则化**：权重衰减、Dropout、Batch Normalization 的原理、使用方法和注意事项。
7. **完整案例**：手机价格分类，涵盖数据准备、模型构建、训练、评估、调优全流程。

通过理论与实践结合，读者应能独立使用 PyTorch 构建和训练全连接神经网络，并根据具体任务进行调优。
